target_setter Documentation

Release 1.0

Chheng Lydiya

Feb 26, 2026

CONTENTS

1 Core Documentation **3**
- 1.1 Introduction . . . 3
- 1.2 Hardware Specification . . . 5
- 1.3 Installation and Configuration . . . 6
- 1.4 Technical Reference . . . 8
- 1.5 Troubleshooting Guide . . . 16

2 Appendices (Technical Background) **19**
- 2.1 Appendix A: Object-Oriented Programming . . . 19
- 2.2 Appendix B: MVC Architecture . . . 89
- 2.3 Appendix C: MVVM Architecture . . . 92
- 2.4 Appendix D: ROS2 Overview . . . 95
- 2.5 Appendix E: Distributed Systems . . . 96
- 2.6 Appendix F: Monolithic Architecture . . . 97
- 2.7 Appendix G: ROS2 Architecture . . . 98
- 2.8 Appendix H: UDP . . . 99
- 2.9 Appendix I: Quaternions . . . 99
- 2.10 Appendix J: Kinematics . . . 100
- 2.11 Appendix K: State Machine . . . 101
- 2.12 Appendix L: PID Control . . . 101
- 2.13 Appendix M: Localization . . . 102
- 2.14 Appendix N: Odometry . . . 103
- 2.15 Appendix O: Waypoint Navigation . . . 103
- 2.16 Appendix P: Complementary Filtering . . . 104
- 2.17 Appendix Q: CAN Bus . . . 105

3 Indices and Tables **107**

Bibliography **109**

Welcome to the official documentation for **Target Setter**. This documentation provides system overview, installation instructions, technical references, and theoretical background used in the project.

CHAPTER

ONE

CORE DOCUMENTATION

1.1 Introduction

1.1.1 Background

Motivation

Following the previous year in Android application development for robotics system, we have been using Android app as the controller for the robots.

However, this approach works with the xbox gamepad that acts as the main controller for our robots, using the buttons and joystick to navigate the robots manually. While this method provides real-time control, it limits automation and precise waypoint navigation. The initial idea of this project originated from Mr.PHOLL Chanphayuk, who proposed an Android-based interface that allows users to interact with the robot through touch inputs on the screen, combined with odometry feedback and a virtual joystick. The concept was intended for interactive tasks, such as robot basketball playing.

After changes in term members and project direction, the focus of the project evolved from manual joystick control to a waypoint-based navigation system. This shift was encouraged by Dr.SRANG Sarot, aiming to reduce dependency on gamepad controllers and to explore a more autonomous navigation approach using Android applications as the main interface.

Problem Statement

Manual control using gamepads or joysticks presents several limitations in robotic navigation systems:

- Continuous use input is required, increasing operator workload
- Precise positioning is difficult due to human reaction time and control sensitivity
- The system lacks scalability for autonomous behaviors

To address these issues, a waypoint navigation system is proposed, where uses can define target positions (waypoints) through an Android application, and the robot navigates to these positions using odometry and orientation data.

However, implementing such a system introduces challenges, including:

- Accurate coordinate transformation between Android interface and the robot's reference frame
- Proper handling of robot orientation (yaw) in navigation calculations
- Reliable communication between the Android application and the robot's ROS2-based control system
- Managing waypoint editing, execution, and return behaviors without causing state conflicts

These challenges motivated the need to redesign the calculation logic and communication protocol to ensure compatibility with ROS2 and improve navigation accuracy.

Literature Review

Waypoint-based navigation is a common approach in mobile robotics, especially in autonomous systems using frameworks such as ROS and ROS2. Previous works and standard robotics literature describe waypoint navigation as a method where a robot moves through a sequence of pre-defined target positions using localization and motion control algorithms.

Odometry-based navigation is widely used for estimating a robot's position by integrating wheel encoder data over time. Although odometry is simple to implement, it is sensitive to drift and orientation errors, particularly in yaw estimation. Many systems combine odometry with higher-level navigation logic to mitigate these issues.

Mobile interfaces for robot control, including Android-based applications, have also been explored as flexible and user-friendly solutions. Touch-based interfaces allow intuitive interaction, but require careful mapping between screen coordinates and real-world robot motion.

This project builds upon these established concepts, focusing on practical implementation rather than theoretical optimization.

Concepts

The waypoint navigation system in this project is based on the following key concepts:

- Waypoint navigation: the robot is instructed to move toward pre-defined target positions instead of being continuously controlled by the user.
- Odometry: the robot's position and orientation are estimated using incremental motion data.
- Yaw orientation: the robot's heading angle is critical for accurate navigation and must be correctly handled in coordinate transformations.
- Client-sever communication: the Android application acts as a client that sends waypoints data to the robot, while the robot processes these commands within a ROS2 environment.

For simplification, the system assumes:

- A planar (2D) environment
- Reliable odometry data without sensor fusion
- Pre-defined motion control handled by the robot's existing control stack

The Android application focuses on target setting, waypoint management, and command transmission, while the robot handles low-level motion execution.

1.1.2 Objective

This project is developed in order to:

- Design and implement Android app to define, edit, and manage navigation waypoints on a pre-defined map
- Develop custom ROS2 control logic to receive waypoint data and execute navigation commands on the robot
- Implement accurate coordinate transformation and scaling between Android interface and the robot's reference frame
- Handle robot orientation (yaw) correctly during waypoint navigation to improve positioning accuracy
- Establish reliable client-server communication between the Android app and the ROS2-based robot system
- Implement waypoint execution features including sequential navigation, editing, and return-to-previous-waypoint behaviors
- Test and evaluate system performance, identifying limitations related to map alignment, yaw accuracy, and state management.

1.1.3 Scope of work

This project covers:

- Android application development (Target_Setter)
- Communication system
- Robotic software development (ROS2)
- System integration and testing

This project does not include:

- Full autonomous robot navigation
- Sensor fusion beyond basic odometry and IMU integration
- Algorithm optimization
- Low-level communication protocol
- Hardware-level motor driver design
- Network security mechanism

1.2 Hardware Specification

This section outlines the hardware components and specifications for the mobile robot platform used in this project. The hardware selection is based on the requirements for waypoint navigation, odometry estimation, and client-server communication with an Android application.

1.2.1 Robot platform

- **Type:** omnidirectional mobile robot with mecanum wheels (holonomic motion)
- **Dimensions:** 0.5 m x 0.5 m x 0.3 m (approximately)

1.2.2 Sensors

The robot is equipped with 4 independent DC motors, driving 4 mecanum wheels with encoders attached for feedback. To compenstate for yaw estimation, an IMU (Inertial Measurement Unit) is included, providing angular velocity and linear acceleration data.

1.2.3 Drive system

- **Motors:** 4 DC motors, each driving on mecanum wheel
- **Motor specifications:** 24 V, 200 W max, integrated encoders
- **Motor drivers:** custom-built smart driver boards, each supporting 2 motors
- **Wheel type:** mecanum 0.06 radius (approximately)
- **Feedback:** quadrature encoders for wheel odometry

1.2.4 Sensors

Inertial Measurement Unit (IMU): provides angular velocity and linear acceleration

- **Model:** HFI-A9
- **Sampling rate:** 100 Hz

Encoder: integrated with each motor; used for odometry estimation

1.2.5 Compute hardware

High-level processor: NVIDIA Jetson Xavier

- **OS:** Ubuntu 20.04
- **Middleware:** ROS2 Foxy
- **Cooling:** cooling fan

Low-level controller: STM32 microcontroller

- Interfaces with motor driver and encoders for real-time control

Communication: CAN bus at 115200 bps

1.2.6 Network infrastructure

- **High-level control:** UDP of Wi-Fi between Android application and Jetson
- **IP configuration:** dynamic IP over DHCP
- **Indoor deployment:** Wi-Fi network required
- **Security:** basic
- **Remote access:** SSH for development and debugging purposes

1.2.7 Power system

Motors: 24 V Li-ion battery pack, DC-DC converter (12 V) for motor drivers

Electronics: 12 V Li-ion battery pack, voltage regulator for Jetson, sensors, and STM32

Safety features:

- Fuses on smart driver boards
- Overcurrent protection
- Emergency stop push button

1.2.8 Safety constraints

Hardware-level safety: fuses and emergency stop

Software-level safety:

- Speed limits enforced in control software
- Communication failure handling
- Watchdog timers for microcontroller to prevent system hang

1.2.9 Notes

1. Ensure CAN bus termination is correct to prevent communication errors
2. Confirm battery voltages before operation; over/under-voltage may damage electronics
3. When adding additional sensors, update ROS2 launch files for proper integration
4. For software testing, always use the emergency stop and verify motor driver fuses

1.3 Installation and Configuration

This section provides a comprehensive guide to installing and configuring the ROS2-based mobile manipulation app. It covers the necessary prerequisites, workspace setup, build instructions, configuration parameters, and tips for running the application effectively.

1.3.1 Prerequisites

- ROS2 installed on Ubuntu (any version is fine)
- Jetson Nano setup with ROS2 and necessary drivers for the robot hardware
- Android Studio installed for app development
- Basic familiarity with ROS2 concepts and Android development
- Network configuration for communication between the Android app and the robot
- Python 3 and necessary ROS2 Python packages for the robot nodes

1.3.2 Workspace Setup

1. Create a ROS2 workspace on the Jetson Nano:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build
source install/setup.bash
```

2. Clone the project repository into the `src` directory:

```
cd ~/ros2_ws/src
git clone <repository_url>
```

3. Build the workspace:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

4. Set up the Android project in Android Studio by importing the project from the cloned repository.

1.3.3 Configuration

- Update the ROS2 node parameters in the configuration files located in the *config* directory of the ROS2 package. This includes setting the correct IP address for communication and any robot-specific parameters.
- Ensure that the Android app's network settings are configured to communicate with the robot's IP address
- Adjust the waypoint navigation parameters such as speed limits, waypoint tolerance, and control gains as needed for your specific robot and environment.

1.3.4 Running the Application

1. Start the ROS2 nodes on the Jetson Nano:

```
source ~/ros2_ws/install/setup.bash
ros2 run <package_name> <package_node> # repeat for all necessary nodes
```

2. Run the Android application from Android Studio on a connected device or emulator. Or you can just open the `.apk` file on the phone directly.
3. Use the app interface to set waypoints and observe the robot's behavior as it navigates to the specified locations.

Attention

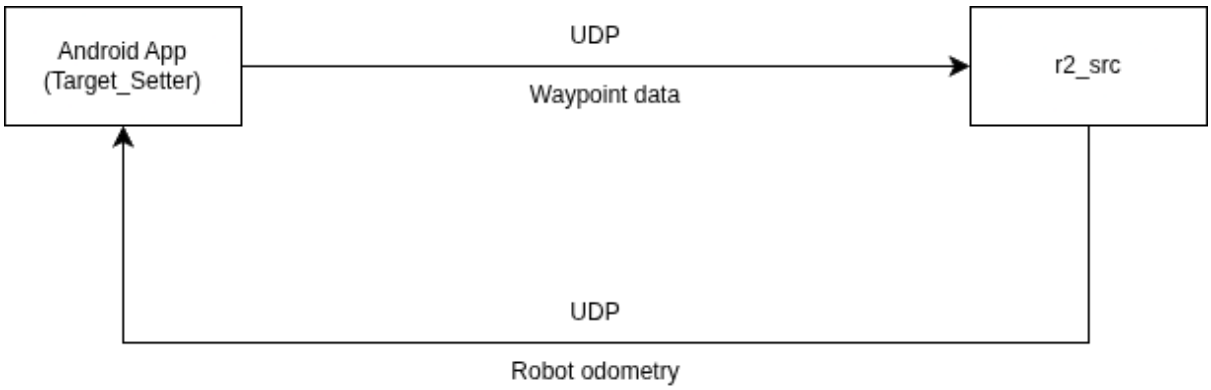
This release doesn't have launch file yet. Launch file will be added in the next update release.

1.3.5 Common Setup Pitfalls

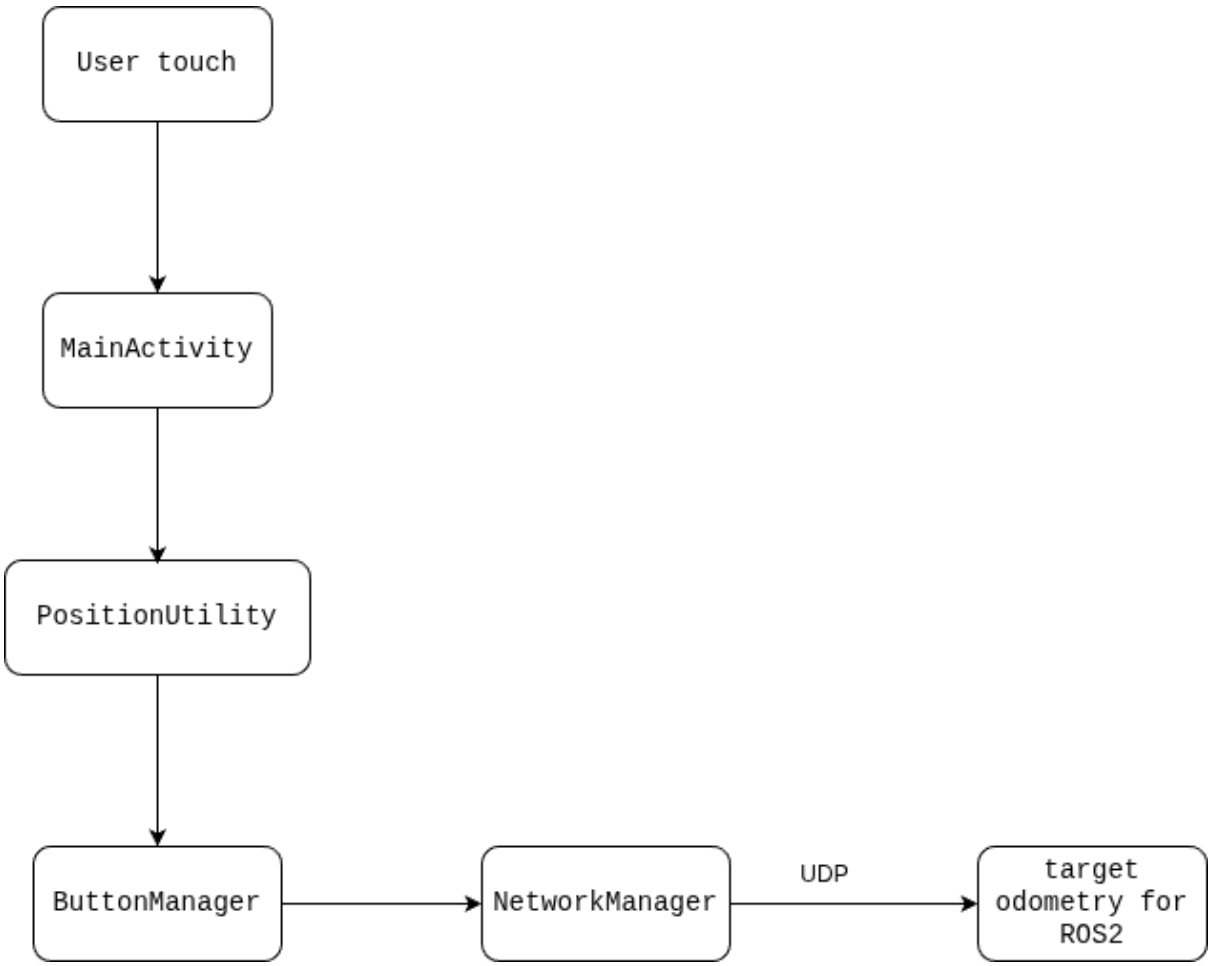
- Incorrect IP address configuration leading to communication failures between the app and the robot.
- Missing dependencies or incorrect ROS2 environment setup on the Jetson Nano.
- Incompatible ROS2 versions between the robot nodes and the app's communication protocol.
- Issues with the robot's hardware drivers or sensor configurations that affect navigation performance.
- Network connectivity issues, such as firewall settings or Wi-Fi interference, that disrupt communication between the app and the robot.

1.4 Technical Reference

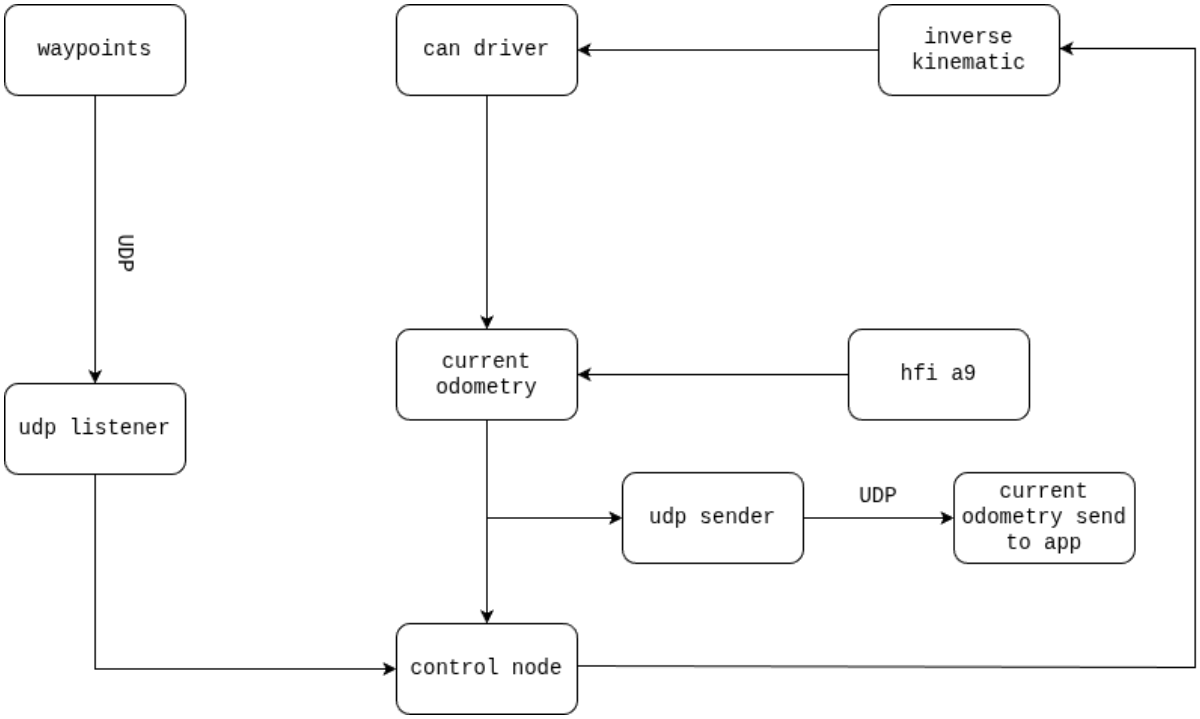
1.4.1 Architecture



Target_Setter app



r2_src



1.4.2 Waypoint queue and state machine

Waypoint queue

- **Waypoint queue:** stores received waypoints from */waypoint*
- **Active target:** currently executing waypoint
- **Visited waypoints:** stack of completed waypoints for return

1.4.3 State machine

- `IDLE`: no active target, waiting for waypoint or return command
- `NAVIGATING`: moving to `active_target`
- `EDIT`: active target is being edited
- `PAUSED`: robot is paused by keyboard or automatic pause
- `RETURNING`: moving to last visited waypoint (return command)

1.4.4 Communication protocol

UDP packet structure

Waypoints packet

- **Purpose:** send a list of waypoints for the robot to follow
- **Frequency:** send when pressing Send in the app

Packet Structure (binary):

Offset	Size (bytes)	Description
0	4	Header (0xAA)
4	4	Number of waypoints (counter)
8	4	Plan ID (version)
12	16 * N	Waypoint (X, Y) in double (8 B)

- N = number of waypoints
- Coordinates are in meters, converted from percentage position
- Byte order: Little Endian

Edit waypoint packet

- **Purpose:** update a previously sent waypoint
- **Frequency:** send when user edits a waypoint via the app

Packet structure (binary):

Offset	Size (bytes)	Description
0	1	Edited flag (1 = edited)
1	4	Waypoint index (1-based)
5	4	Plan ID
9	8	New X coordinate (double)
17	8	New Y coordinate (double)

Return message packet

- **Purpose:** send a single boolean signal to the robot
- **Frequency:** send when return button is pressed

Packet structure (binary):

Offset	Size (bytes)	Description
0	1	Edited flag (0=false,1=true)

Odometry data reception

- **Purpose:** receives current robot odometry
- **Frequency:** 10Hz

Packet structure (binary):

Offset	Size (bytes)	Description
0	8	X position (double)
8	8	Y position (double)
16	8	Yaw (double, radians)

- Byte order: Little Endian
- Latest odometry stored in memory for waypoint calculations

UDP data flow

Sending data

- App uses `NetworkManager.sendWaypoints()`, `sendEdit()`, and `sendMesage()`
- Waypoints are converted to binary packets before sending
- Each send operation is done in a separate thread
- Plan ID differentiates waypoint plans
- Packets sent via UDP to robot IP/port (default 5050)

Receiving data

- Persistent UDP socket listens (`startReceiving(port)`)
- Robot sends odometry at 10Hz (X, Y, Yaw)
- Incoming packets converted using Little Endian byte order
- Latest odometry stored in `latestOdometry`

Processing data

When adding waypoint:

- Convert touch position to normalized coordinates
- Compute target X, Y in meters
- Add to waypoint list, update UI
- Send waypoint via UDP

When editing waypoint:

- Specify index and new coordinates
- Update list and redraw UI
- Send edit packet via UDP with plan ID

Return message:

- Single-byte packet sent to trigger return behavior

Odometry:

- Display robot position
- Compute accurate target positions

1.4.5 Coordinate convention and control logic

Coordinate flow

Touch → UI → App → Robot

```
Touch Event (screen pixels)
   ↓ absolutePosition()
Absolute Coordinates relative to boundaryView
   ↓ normalizedPosition()
Normalized Coordinates (0-1)
   ↓ percentagePosition()
Percentage Coordinates (0-100%)
   ↓ targetPosition()
Global Frame (meters) → UDP → Robot
```

Robot → App (Odometry)

```
Odometry Packet (X, Y, yaw in meters) → inputPosition()
   ↓ metersToPxX/Y
Pixels on UI → Display robot position
```

Frame definition

Local frame:

- Origin: the center of the robot
- +x: forward
- +y: left
- +yaw: counter-clockwise

Global frame:

- Origin: top-left of the screen on the target_setter app
- +x: rightward on the screen on the target_setter app
- +y: downward on the screen on the target_setter app
- Waypoints that is sent to the robot is in global frame

Units:

On the robot and the data sent from app:

- x: meters
- y: meters
- yaw: radians (data from app only send x and y, yaw must be infer from them)

- Vx: m/s
- Vy: m/s
- Wz: rad/s

On the app:

- x: percentage
- y: percentage

Touch input conversion

Absolute position (touch):

```
double absoluteX = rawX - boundaryLeft;
double absoluteY = rawY - boundaryTop;
```

- Converts raw screen coordinate to coordinate relative to *boundaryView*
- Clamped to boundary edges

Normalized position (0-1):

```
double normalizedX = absoluteX / screenWidth;
double normalizedY = absoluteY / screenHeight;
```

- Represents position as fraction of the screen

Percentage position (0-100%):

```
double percentageX = normalizedX * 100.0;
double percentageY = normalizedY * 100.0;
```

- Used as intermediate format for calculation into world meters
- Checked against 0-100% boundary

Global meters (for UDP):

```
double targetX = (percentageX / 100.0) * GAME_FIELD_X;
double targetY = (percentageY / 100.0) * GAME_FIELD_Y;
```

- Converts to meters based on field dimension
- Result is sent to robot

Robot odometry (UI display):

```
double inputX = odometryData[0] * (screenWidth / GAME_FIELD_X);
double inputY = odometryData[1] * (screenHeight / GAME_FIELD_Y);
```

- Converts meters to pixels for UI display
- Clamped boundary using *validateBoundary()*

1.4.6 Control logic

PD controller

- **Position errors:**

computed in global frame

```
current_x_error_g = self.desired_x - self.current_x
current_y_error_g = self.desired_y - self.current_y

current_yaw_p_error = self.normalize_angle(self.desired_yaw - self.current_yaw)
```

- **Derivative errors:**

```
current_yaw_d_error = (current_yaw_p_error - self.previous_yaw_p_error)/self.dt
current_x_d_error = (current_x_error_g - self.previous_x_p_error)/self.dt
current_y_d_error = (current_y_error_g - self.previous_y_p_error)/self.dt
```

- **Complementary filtering:**

apply complementary filter on D term to smooth the movement

```
current_x_d_error = ALPHA_D * current_x_d_error + (1 - ALPHA_D) * self.previous_x_d_
↪error
current_y_d_error = ALPHA_D * current_y_d_error + (1 - ALPHA_D) * self.previous_y_d_
↪error
current_yaw_d_error = ALPHA_D * current_yaw_d_error + (1 - ALPHA_D) * self.previous_
↪yaw_d_error
```

- **PD controller:**

```
linear_vel_x_g = self.k_p_linear * current_x_error_g + self.k_d * current_x_d_error
linear_vel_y_g = self.k_p_linear * current_y_error_g + self.k_d * current_y_d_error
angular_vel_z_g = self.k_p_yaw * current_yaw_p_error + self.k_d_yaw * current_yaw_d_
↪error
```

- **Frame conversion:**

```
linear_vel_x_l =  math.cos(self.current_yaw) * current_x_error_g + math.sin(self.
↪current_yaw) * current_y_error_g
linear_vel_y_l = -math.sin(self.current_yaw) * current_x_error_g + math.cos(self.
↪current_yaw) * current_y_error_g
```

- Units must match: meters (x, y), radians (yaw), m/s (velocities)
- Linear velocity in robot frame, angular velocity around z-axis
- Low-pass filtering applied on derivative term
- Return *linear_vel_x*, *linear_vel_y*, and *angular_vel_z*

Dead reckoning

If the velocity is less than error threshold, overwrite it with 0

Speed limit

If velocity exceeds maximum speed, overwrite it with maximum speed

1.4.7 Path planning

Arrivel detection

- Target reached if: - Distance < `goal_tolerance_pos = 0.05m` - Remains withitn tolerance for `0.5s`
- Arrival time accumulated using `dt` from timer callback

Waypoint callback

- Store each waypoint in a dictionary
- Append it into the queue
- Copy the waypoint in the queue to `active_target` to execute

Waypoints execution

- if `active_target` does not exist, and there's waypoint in the queue, pop the queue, then reset robot state to go to the waypoint that's being popped
- if return is triggered, and there's visited waypoint, command the robot to go the visited waypoint
- Otherwise, stop the robot

Waypoint update

- Store the edited waypoint in a dictionary
- If the edited waypoint is active, use complementary to smooth the robot movement when update to prevent jumping suddenly

```
self.active_target['x'] = (1 - ALPHA) * self.active_target['x'] + ALPHA * update_wp[
↪'updated_x']
self.active_target['y'] = (1 - ALPHA) * self.active_target['y'] + ALPHA * update_wp[
↪'updated_y']
```

- Otherwise, just update the waypoint as normal

Pause

- Robot pauses motion at each target
- If distance from the robot to the target is less than tolerance, stop the robot
- If the elapsed time during pause exceeds pause duration, command the robot to go to the next waypoint

Return

- Update the return flag

1.4.8 Odometry handling

Frame conversion

- Calculate velocity in local frame

```
linear_vel_x_local = (self.R/4) * (self.motor_vel[0] + self.motor_vel[1] + self.motor_
↪vel[2] + self.motor_vel[3])
linear_vel_y_local = (self.R/4) * (-self.motor_vel[0] + self.motor_vel[1] + self.
↪motor_vel[2] - self.motor_vel[3])
w_z = (self.R * (-self.motor_vel[0] + self.motor_vel[1] - self.motor_vel[2] + self.
↪motor_vel[3]))/(2 * (self.l_x + self.l_y))
```

- Convert them to global frame

```
v_x_global = math.cos(self.current_yaw) * linear_vel_x_local - math.sin(self.current_
↪yaw) * linear_vel_y_local
v_y_global = math.sin(self.current_yaw) * linear_vel_x_local + math.cos(self.current_
↪yaw) * linear_vel_y_local
```

- Integrate velocity to get position

```
self.x += v_x_global * self.dt
self.y += v_y_global * self.dt
```

Offset

```
self.x_odom = (self.x - self.x_start) - self.frame_offset_x
self.y_odom = (self.y - self.y_start) - self.frame_offset_y
```

Yaw

- Calculate yaw from quaternion

```
def calculate_yaw(self, q_x, q_y, q_z, q_w):
    siny_cosp = 2 * (q_w * q_z + q_x * q_y)
    cosy_sinp = 1 - 2 * (q_y**2 + q_z**2)
    return math.atan2(siny_cosp, cosy_sinp)
```

- Normalize yaw

```
def normalize_angle(self, a):
    return math.atan2(math.sin(a), math.cos(a))
```

- Offset yaw

```
self.current_yaw = (self.yaw - self.yaw_start) + 2 * math.pi * self.overflow_counter
```

- Convert to quaternion to publish

```
def publish_yaw(self, yaw):
    q = Quaternion()
    q.w = math.cos(yaw/2)
    q.x = 0.0
    q.y = 0.0
    q.z = math.sin(yaw/2)
    return q
```

1.4.9 Inverse Kinematic

```
V_wheel = [
Vx/self.R - Vy/self.R - omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R + Vy/self.R + omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R + Vy/self.R - omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R - Vy/self.R + omega_z*(self.lx + self.ly)/(2.0*self.R)
]
```

1.5 Troubleshooting Guide

1.5.1 Communication issues

Symptom: robot doesn't repond to app command

Wi-Fi connectivity issues:

- Check that the robot is connected to the correct Wi-Fi network and has a valid IP address
- Verify IP address of the robot matches the one configured in the app
- Use `ifconfig` on the Jetson to check network interfaces and IP configuration

UDP packets not received:

- Confirm the correct port (default `5050`) is used
- Ensure packets are sent in **Little Endian** format

Firewall blocking UDP:

- Disable firewall or allow UDP traffic on port 5050

Symptom: waypoints not updating/edits ignored

Possible causes and solutions:

- Plan ID mismatch between app and robot
- Edited waypoint index out of range
- Packet corruption: check Little Endian encoding and packet size

1.5.2 Motor/driver issues

Symptom: robot does not move or moves incorrectly

Possible causes and solutions:

- Motor driver fuse blown: replace fuse and test
- Motors not powered: verify 24 V motor battery and DC-DC converter
- Encoder feedback lost: check encoder connections to STM32
- Wheels misaligned: check mecanum wheel orientation
- Incorrect velocities from controller: verify PD controller gains

Symptom: robot moves jerkily or overshoots targets

- Derivative term too noisy: verufy `ALPHA_D` filtering
- Low update frequency: check odometry reception at 10 Hz
- Active waypoint updates cause jumps: ensure complementary smoothing is applied

1.5.3 Sensor issues

Symptom: odometry is incorrect/robot drifts

- Checks IMU calibration and orientation
- Verify encoder counts correspond to expected wheel rotation
- Confirm yaw offset calculation (`yaw_start`, `overflow_counter`)
- Dead reckoning integration might accumulate error: check `dt` values

Symptom: robot reports zero movement despite motor command

- Motor velocities not reaching STM32: verify CAN bus communication
- Verify low-level motor control code maaps velocities correctly to wheels

1.5.4 App issues

Symptom: robot position not displayed correctly

- Verify coordinate conversions from meters to pixels (`metersToPxX/Y`) are correct
- Check `latestOdometry` is being updated from UDP socket
- Confirm screen boundary and field dimensions (`GAME_FIELD_X/Y`) match robot workspace

Symptom: touch input does not map to correct target

- Check touch, normalized, percentage, global meter conversion
- Ensure `boundaruView` dimensions match the UI

1.5.5 Safety/emergency stop

Symptom: robot does not stop when emergency button pressed

- Verify emergency stop wiring and fuses are intact
- Ensure software checks for emergency stop signal are active
- Test manually by cutting power to confirm hardware response

1.5.6 PD controller/motion control

Symptom: robot oscillates around waypoint

- Decreases `k_p_linear` or `k_p_yaw` gains
- Increase derivative damping (`ALPHA_D`)
- Verify derivative errors are computed correctly using `dt`

Symptom: robot fails to reach waypoint

- Check `goal_tolerance_pos` and pause duration
- Confirm `active_target` queue is populated and not empty
- Make sure velocity limits allow the robot to reach target within constraints

1.5.7 Low-level hardware checks

- **Jetson:** check power LED, CPU usage, and fan
- **STM32:** verify firmware is running; check CAN message counts
- **Batteries:** measure voltage under load; replace if below threshold
- **Motor drivers:** check for overcurrent events or thermal shutdowns

Attention

1. Always start troubleshooting from connectivity, sensors, motion, control logic
2. Use logs from app (`sendWaypoints()`, `latestOdometry`) to verify packet flow
3. Test motor and wheel functionality before running autonomous commands
4. Keep fuses, emergency stop, and emergency protocols ready during testing

CHAPTER

TWO

APPENDICES (TECHNICAL BACKGROUND)

2.1 Appendix A: Object-Oriented Programming

2.1.1 Introduction

Object-oriented programming (OOP) addresses limitations of procedural paradigms when managing complexity in larger systems. While procedural programming organizes code around functions, OOP organizes code around objects, which encapsulate data (attributes) and behavior (methods). This approach promotes:

- **Modular design:** objects serve as self-contained modules
- **Code reuse:** inheritance and composition enable reuse of behavior
- **Data safety:** encapsulation helps protect internal state
- **Scalability:** modular components can be extended or replaced without affecting the whole system
- **Natural modeling:** objects map naturally to real-world entities, making designs easier to reason about

Historical influences

OOP has evolved through decades of research and language design:

- **Simula (1960s):** introduced classes and objects
- **Smalltalk (1970s):** popularized OOP concepts, including message passing and dynamic typing
- **C++ (1980s):** extended C with classes, inheritance, and polymorphism
- **Java (1990s):** designed as an OOP language emphasizing portability and safety
- **Python (1990s):** supports multiple paradigms, including OOP, with a dynamic type system
- **C# (2000s):** modern OOP language integrated with the `.NET` framework

Core OOP principles

- **Encapsulation:** bundle data and the methods that operate on it in a single unit (class), and restrict access to internal components when appropriate.
- **Inheritance:** create new classes based on existing ones to enable reuse and extension.
- **Polymorphism:** treat different classes uniformly through a common interface (method overriding, interfaces, or duck typing).
- **Abstraction:** expose only the necessary features of an object while hiding implementation details.

OOP aligns with design principles such as SOLID and common design patterns (singleton, factory, observer), which formalize reusable solutions to recurrent design problems.

2.1.2 Type systems and object models

OOP languages differ by type system and object model:

- **Class-based vs. Prototype-based:** class-based languages use classes as blueprints; prototype-based languages (e.g., JavaScript) create objects from other objects.
- **Static vs. Dynamic typing:** static typing requires compile-time type information; dynamic typing resolves types at runtime.
- **Single vs. Multiple inheritance:** some languages (Java, C#) restrict class inheritance to a single parent but allow interfaces; others (C++, Python) support multiple inheritance.
- **Interface-based vs. Class-based polymorphism:** languages use interfaces, virtual methods, or duck typing to achieve polymorphism.
- **Memory management:** some languages (C++) require explicit memory management; others (Java, Python, JavaScript) use automatic garbage collection.

Language-specific notes

- **Java:** strong encapsulation, robust type system, single class inheritance with interfaces, automatic garbage collection.
- **C++:** manual and RAII-style resource management options, multiple inheritance, fine-grained control over performance.
- **Python:** dynamic typing, multiple inheritance, flexible object model, garbage-collected.
- **JavaScript:** prototype-based, dynamic typing, flexible object creation and modification.
- **C#:** modern language features, strong typing, single inheritance with interfaces, integrated with `.NET`.

Object model differences

The object model affects how objects are created and interact. For example: - Java: class-based instantiation, garbage collection, single inheritance with interfaces for polymorphism. - C++: class-based, manual or RAII-managed resources, multiple inheritance. - Python: class-based, dynamic typing, multiple inheritance, runtime flexibility. - JavaScript: prototype-based, dynamic object composition and modification.

2.1.3 Scope of this appendix

This appendix focuses on practical OOP guidance relevant to this project:

- ROS2 nodes implemented in Python (Jetson)
- Android app components implemented in Java

We emphasize:

- structuring classes for modular, reusable components
- designing maintainable and extensible class hierarchies
- applying OOP principles to manage robot subsystems

C++ and JavaScript OOP concepts are not covered in detail here.

2.1.4 Foundational OOP concepts

Class

A class is a template for objects. It defines attributes (state) and methods (behavior). Classes enable modular design and reuse.

```python
class MotorController:
    def __init__(self, motor_id):
        self.motor_id = motor_id
        self.speed = 0

    def set_speed(self, speed):
        self.speed = speed
```

```java
public class MotorController {
    private int motorId;
    private int speed;

    public MotorController(int motorId) {
        this.motorId = motorId;
        this.speed = 0;
    }

    public void setSpeed(int speed) {
        this.speed = speed;
    }
}
```

Object

An object is an instance of a class with its own state and behavior.

```python
motor1 = MotorController(1)
motor2 = MotorController(2)

motor1.set_speed(100)
motor2.set_speed(150)
```

```java
MotorController motor1 = new MotorController(1);
MotorController motor2 = new MotorController(2);

motor1.setSpeed(100);
motor2.setSpeed(150);
```

Attributes

Attributes define an object's state.

```python
class Sensor:
    def __init__(self, sensor_id):
        self.sensor_id = sensor_id
        self.value = 0
```

```java
public class Sensor {
    private int sensorId;
    private int value;

    public Sensor(int sensorId) {
        this.sensorId = sensorId;
        this.value = 0;
    }
}
```

- Prefer restricting direct access to attributes (private/protected) when appropriate to enforce invariants and reduce coupling.
- Use getters/setters or properties to control and validate updates to internal state.

Methods

Methods define behavior and operate on an object's state.

```python
class WaypointNavigator:
    def __init__(self):
        self.waypoints = []

    def add_waypoint(self, x, y):
        self.waypoints.append((x, y))
```

```java
public class WaypointNavigator {
    private List<Point> waypoints;
    public WaypointNavigator() {
        this.waypoints = new ArrayList<>();
    }
}
```

Methods encapsulate operations on internal state and support interaction between objects.

Instance and class members

- **Instance members** belong to individual objects.
- **Class members** (static/class variables) are shared across all instances.

```python
class Robot:
    total_robots = 0  # class variable

    def __init__(self, name):
        self.name = name  # instance variable
        Robot.total_robots += 1

    @classmethod
    def get_total_robots(cls):
        return cls.total_robots
```

```java
public class Robot {
    private static int totalRobots = 0; // class variable
    private String name; // instance variable

    public Robot(String name) {
        this.name = name;
        totalRobots++;
    }

    public static int getTotalRobots() {
        return totalRobots;
    }
}
```

Object state and behavior

- State is defined by attributes; behavior is defined by methods that operate on that state.

```
motor1 = MotorController(1)
motor1.set_speed(100)  # Changes motor1's state
```

```
MotorController motor1 = new MotorController(1);
motor1.setSpeed(100); // Changes motor1's state
```

Message passing

Objects interact by calling methods on one another. This keeps components decoupled and promotes clear interfaces.

```
class SensorManager:
    def __init__(self):
        self.sensors = []

    def add_sensor(self, sensor):
        self.sensors.append(sensor)

    def read_all_sensors(self):
        return [sensor.value for sensor in self.sensors]
```

```
public class SensorManager {
    private List<Sensor> sensors;
    public SensorManager() {
        this.sensors = new ArrayList<>();
    }
    public void addSensor(Sensor sensor) {
        this.sensors.add(sensor);
    }
    public List<Integer> readAllSensors() {
        List<Integer> values = new ArrayList<>();
        for (Sensor sensor : sensors) {
            values.add(sensor.getValue());
        }
        return values;
    }
}
```

2.1.5 Core concepts

This section covers encapsulation, inheritance, polymorphism, and abstraction with practical guidance for ROS2 (Python) and Android (Java) components.

Encapsulation

Encapsulation protects an object's internal state and exposes a minimal, well-documented interface. Use access modifiers (Java) or naming conventions/properties (Python) to enforce invariants and validate inputs in setter methods or properties.

```
class MotorController:
    def __init__(self, motor_id):
        self._motor_id = motor_id
        self._speed = 0
```

(continues on next page)

(continued from previous page)

```
    def set_speed(self, speed):
        if speed < 0:
            raise ValueError("speed must be non-negative")
        self._speed = speed

    def get_speed(self):
        return self._speed
```

```
public class MotorController {
    private final int motorId;
    private int speed;

    public MotorController(int motorId) {
        this.motorId = motorId;
        this.speed = 0;
    }

    public void setSpeed(int speed) {
        if (speed < 0) throw new IllegalArgumentException("speed must be non-negative
↪");
        this.speed = speed;
    }

    public int getSpeed() {
        return this.speed;
    }
}
```

Benefits

- prevents invalid state transitions
- reduces coupling between components
- simplifies testing and reasoning about behavior

Abstraction

Abstraction exposes only the information needed by clients. Keep complex algorithms behind a simple method or interface and document expected behavior, side effects, and performance characteristics.

```
class Robot:
    def move_to(self, x, y):
        """High-level API: performs path planning and motor control internally."""
        # Implementation details hidden
        pass
```

```
public class Robot {
    public void moveTo(int x, int y) {
        // High-level API; internal planning and actuation hidden
    }
}
```

Inheritance

Use inheritance to express an "is-a" relationship. Prefer composition over inheritance when behavior can be composed from smaller components. Avoid deep inheritance chains; prefer small, well-defined base classes or interfaces.

```python
class Vehicle:
    def move(self):
        raise NotImplementedError

class Robot(Vehicle):
    def move(self):
        # robot-specific movement
        pass
```

```java
public class Vehicle {
    public void move() {
        // default or abstract behavior
    }
}
```

Polymorphism

Polymorphism lets different implementations be treated through a common interface. Use interfaces (Java) or duck typing/abstract base classes (Python) to decouple consumers from concrete implementations.

```python
class Sensor:
    def read_value(self):
        raise NotImplementedError

class Temperature(Sensor):
    def read_value(self):
        return 25  # example value

class Pressure(Sensor):
    def read_value(self):
        return 1013
```

```java
public interface Sensor {
    int readValue();
}

public class Temperature implements Sensor {
    @Override
    public int readValue() {
        return 25; // example
    }
}

public class Pressure implements Sensor {
    @Override
    public int readValue() {
        return 1013;
    }
}
```

2.1.6 Object lifecycle and structure

Construction and destruction of objects, as well as the structure of classes and their relationships, are fundamental aspects of OOP. We will discuss how to properly initialize objects using constructors, manage resources with destructors (where applicable), and design class hierarchies that promote maintainability and extensibility. We will also cover advanced OOP concepts such as method overloading, method overriding, dynamic binding, and design patterns that can be applied to our project to create a robust and scalable software architecture.

Constructor

A constructor is a special method used to initialize objects when they are created. It typically sets up the initial state of the object by assigning values to its attributes.

```python
class Robot:
    def __init__(self, name):
        self.name = name  # Initialize the robot's name
        self.position = (0, 0)  # Initialize the robot's position
```

```java
public class Robot {
    private String name; // Robot's name
    private Point position; // Robot's position

    public Robot(String name) {
        this.name = name; // Initialize the robot's name
        this.position = new Point(0, 0); // Initialize the robot's position
    }
}
```

Tip

- Keep constructors lightweight; avoid long-running operations
- Initialize all critical attributes to a known state
- In ROS2, avoid ROS communications in constructors; use a separate `initialize()` method if needed

Destructor

A destructor is a special method that is called when an object is destroyed or goes out of scope. It is used to perform cleanup operations, such as releasing resources or closing connections.

```python
class Robot:
    def __del__(self):
        # Perform cleanup operations, such as closing connections
        pass
```

Python uses garbage collection to manage memory, so destructors are not commonly used. However, you can define a `__del__` method to perform cleanup if necessary.

Java has a garbage collector that automatically manages memory, so it does not have a destructor like C++. Instead, you can use the `finalize` method or implement the `AutoCloseable` interface for resource management. However, relying on `finalize` is generally discouraged in Java due to unpredictability, and using `AutoCloseable` with try-with-resources is the recommended approach for managing resources.

Constructor overloading

Constructor overloading allows a class to have multiple constructors with different parameter lists. This provides flexibility in object instantiation, allowing objects to be created with different initial states.

```python
class Robot:
    def __init__(self, name, position=(0, 0)):
        self.name = name
        self.position = position
```

```java
public class Robot {
    private String name;
    private Point position;

    public Robot(String name) {
        this.name = name;
        this.position = new Point(0, 0);
    }

    public Robot(String name, Point position) {
        this.name = name;
        this.position = position;
    }
}
```

Copy constructor

Copy constructors allow for the creation of a new object as a copy of an existing object. This is particularly useful when you want to create a new instance with the same state as an existing instance.

```python
import copy

motor2 = copy.deepcopy(motor1)  # Create a deep copy of motor1
```

```java
public MotorController(MotorController original) {
    this.motorId = original.motorId;
    this.speed = original.speed;
}
```

Attention

- In Python, use the `copy` module for shallow and deep copying of objects.
- In Java, implement a copy constructor or use the `Cloneable` interface for object copying. Be cautious with mutable objects to avoid unintended side effects.

Initialization block

Java supports initialization blocks, which are used to initialize instance variables. They are executed when an instance of the class is created, before the constructor is called. This can be useful for common initialization code that is shared across multiple constructors.

```java
public class Robot {
    private String name;
    private Point position;

    // Initialization block
    {
        this.position = new Point(0, 0); // Default position for all robots
    }
```

(continues on next page)

(continued from previous page)

```
    public Robot(String name) {
        this.name = name; // Initialize the robot's name
    }

    public Robot(String name, Point position) {
        this.name = name; // Initialize the robot's name
        this.position = position; // Override default position if provided
    }
}
```

Object instantiation

Object instantiation is the process of creating an instance of a class. In Python, this is done by calling the class as if it were a function, which invokes the constructor to initialize the object. In Java, you use the `new` keyword followed by the class name and parentheses to create a new instance.

```
robot1 = Robot("Robo1")  # Create a new Robot instance with name "Robo1"
robot2 = Robot("Robo2", (10, 10))  # Create a new Robot instance with name "Robo2"␣
↪and position (10, 10)
```

```
Robot robot1 = new Robot("Robo1"); // Create a new Robot instance with name "Robo1"
Robot robot2 = new Robot("Robo2", new Point(10, 10)); // Create a new Robot instance␣
↪with name "Robo2" and position (10, 10)
```

Memory allocation

In Python, memory allocation for objects is handled automatically by the Python memory manager and garbage collector. When an object is created, memory is allocated for it on the heap, and when it is no longer needed, the garbage collector will free that memory.

2.1.7 Keywork and modifiers

`this` or `self` keyword

`this` in Java and `self` in Python refer to the current instance of the class. They are used to access instance variables and methods from within the class.

`super` keyword

The `super` keyword is used in subclasses to refer to the superclass. It allows you to call methods and access attributes of the superclass, which is particularly useful when overriding methods.

```
class AdvancedRobot(Robot):
    def __init__(self, name, position=(0, 0), sensors=[]):
        super().__init__(name, position)  # Call the superclass constructor
        self.sensors = sensors  # Additional attribute for AdvancedRobot
```

```
public class AdvancedRobot extends Robot {
    private List<Sensor> sensors;
    public AdvancedRobot(String name, Point position, List<Sensor> sensors) {
        super(name, position); // Call the superclass constructor
        this.sensors = sensors; // Additional attribute for AdvancedRobot
    }
}
```

Access modifiers

- **Public:** accessible from anywhere
- **Private:** accessible only within the class
- **Protected:** accessible within the class and its subclasses

In Python, there are no strict access modifiers, but by convention, a single underscore prefix (e.g., `_attribute`) indicates that an attribute is intended for internal use (`protected`), while a double underscore prefix (e.g., `__attribute`) triggers name mangling to make it harder to access from outside the class (`private`).

```
class Robot:
    def __init__(self, name):
        self.name = name  # public attribute
        self._internal_state = "active"  # protected attribute
        self.__secret_code = "1234"  # private attribute

class AdvancedRobot(Robot):
    def reveal_secret(self):
        return self._Robot__secret_code  # Accessing the private attribute using name␣
↪mangling
```

In Java, you can use the `public`, `private`, and `protected` keywords to control access to class members. Additionally, Java has a default (package-private) access level when no modifier is specified, which allows access within the same package.

```
public class Robot {
    public String name; // public attribute
    protected String internalState; // protected attribute
    private String secretCode; // private attribute

    public Robot(String name) {
        this.name = name;
        this.internalState = "active";
        this.secretCode = "1234";
    }
}
```

`final` or `const` keyword

The `final` keyword in Java and the `const` keyword in C++ are used to indicate that a variable's value cannot be changed once it has been assigned. In Java, you can declare a variable as `final` to make it a constant, and in C++, you can use `const` to achieve the same effect. This is useful for defining constants or ensuring that certain values remain unchanged throughout the program.

```
public class Constants {
    public static final double PI = 3.14159; // Constant value for PI
}
```

In Python, there is no built-in keyword for constants, but by convention, variables that are meant to be constants are typically defined in uppercase letters (e.g., `PI = 3.14`) to indicate that they should not be modified.

```
PI = 3.14159  # Constant value for PI (by convention, not enforced by the language)
```

`abstract` keyword

The `abstract` keyword is used in Java to declare a class as abstract, which means that it cannot be instantiated on its own and must be subclassed. An abstract class can contain abstract methods, which are declared without an implementation and must be implemented by subclasses. This allows for defining a common interface for a group of related classes while allowing each subclass to provide its own specific implementation.

```java
public abstract class Shape {
    public abstract double area(); // Abstract method to be implemented by subclasses
}

public class Circle extends Shape {
    private double radius;
    public Circle(double radius) {
        this.radius = radius;
    }
    @Override
    public double area() {
        return Math.PI * radius * radius; // Implementation of the abstract method
    }
}
```

In Python, you can achieve similar functionality using the `abc` module to create abstract base classes and abstract methods.

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass  # Abstract method to be implemented by subclasses

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius  # Implementation of the abstract␣
↪method
```

`virtual` keyword

The `virtual` keyword is used in C++ to indicate that a method can be overridden in a derived class. This allows for dynamic dispatch, where the method that gets called is determined at runtime based on the actual type of the object, rather than the type of the reference. This is a key feature of polymorphism in C++.

```cpp
class Vehicle {
public:
    virtual void move() {
        // Default movement behavior
    }
};

class Robot : public Vehicle {
public:
    void move() override {
        // Robot-specific movement behavior
    }
};
```

In Python and Java, all methods are virtual by default, meaning they can be overridden in subclasses without needing a specific keyword. However, in Java, you can use the `final` keyword to prevent a method from being overridden.

```java
public class Vehicle {
    public void move() {
        // Default movement behavior
    }
}

public class Robot extends Vehicle {
    @Override
    public void move() {
        // Robot-specific movement behavior
    }
}
```

2.1.8 Relationships between classes

A class can have a relationship with another class, such as association, aggregation, composition, dependency, realization, and generalization. These relationships define how classes interact with each other and how they are structured in a class hierarchy. We will discuss these relationships in detail and provide examples of how they are used in our project to create a well-designed and maintainable codebase.

Association

Association is a relationship between two classes where one class uses or interacts with another class. It represents a "has-a" relationship, where one class contains a reference to another class.

```python
class Sensor:
    def __init__(self, sensor_id):
        self.sensor_id = sensor_id

class Robot:
    def __init__(self, name):
        self.name = name
        self.sensor = Sensor(1)  # Robot has a Sensor (association)
```

```java
public class Sensor {
    private int sensorId;
    public Sensor(int sensorId) {
        this.sensorId = sensorId;
    }
}

public class Robot {
    private String name;
    private Sensor sensor; // Robot has a Sensor (association)

    public Robot(String name) {
        this.name = name;
        this.sensor = new Sensor(1); // Initialize the sensor
    }
}
```

Aggregation

Aggregation is a special form of association that represents a "whole-part" relationship, where one class (the whole) contains another class (the part), but the part can exist independently of the whole.

```python
class Sensor:
    def __init__(self, sensor_id):
        self.sensor_id = sensor_id

class Robot:
    def __init__(self, name, sensor):
        self.name = name
        self.sensor = sensor  # Robot has a Sensor (aggregation)
```

```java
public class Sensor {
    private int sensorId;
    public Sensor(int sensorId) {
        this.sensorId = sensorId;
    }
}

public class Robot {
    private String name;
    private Sensor sensor; // Robot has a Sensor (aggregation)

    public Robot(String name, Sensor sensor) {
        this.name = name;
        this.sensor = sensor; // Initialize the sensor
    }
}
```

Composition

Composition is a stronger form of aggregation that represents a "whole-part" relationship, where one class (the whole) contains another class (the part), and the part cannot exist independently of the whole.

```python
class Sensor:
    def __init__(self, sensor_id):
        self.sensor_id = sensor_id

class Robot:
    def __init__(self, name):
        self.name = name
        self.sensor = Sensor(1)  # Robot has a Sensor (composition)
```

```java
public class Sensor {
    private int sensorId;
    public Sensor(int sensorId) {
        this.sensorId = sensorId;
    }
}

public class Robot {
    private String name;
    private Sensor sensor; // Robot has a Sensor (composition)

    public Robot(String name) {
        this.name = name;
        this.sensor = new Sensor(1); // Initialize the sensor
    }
}
```

Dependency

Dependency is a relationship where one class depends on another class to function. It represents a "uses-a" relationship, where one class uses another class as part of its functionality.

```python
class Sensor:
    def __init__(self, sensor_id):
        self.sensor_id = sensor_id

class Robot:
    def __init__(self, name):
        self.name = name

    def read_sensor(self, sensor):
        return sensor.sensor_id  # Robot uses Sensor (dependency)
```

```java
public class Sensor {
    private int sensorId;
    public Sensor(int sensorId) {
        this.sensorId = sensorId;
    }

public class Robot {
    private String name;
    public Robot(String name) {
        this.name = name;
    }

    public int readSensor(Sensor sensor) {
        return sensor.getSensorId(); // Robot uses Sensor (dependency)
    }
}
```

Realization

Realization is a relationship where one class implements an interface defined by another class. It represents a "implements" relationship, where a class provides the implementation for the methods defined in an interface.

```java
public interface Sensor {
    int readValue(); // Interface method
}

public class TemperatureSensor implements Sensor {
    @Override
    public int readValue() {
        // Return temperature reading
    }
}

public class PressureSensor implements Sensor {
    @Override
    public int readValue() {
        // Return pressure reading
    }
}
```

In Python, you can achieve a similar effect using abstract base classes and the `abc` module to define interfaces and their implementations.

```python
from abc import ABC, abstractmethod

class Sensor(ABC):
    @abstractmethod
    def read_value(self):
        pass  # Interface method

class TemperatureSensor(Sensor):
    def read_value(self):
        # Return temperature reading
        pass
```

Generalization

Generalization is a relationship where one class is a more general version of another class. It represents an "is-a" relationship, where a subclass is a specialized version of a superclass.

```python
class Vehicle:
    def move(self):
        pass

class Robot(Vehicle):
    def move(self):
        # Implement robot-specific movement logic
        pass
```

```java
public class Vehicle {
    public void move() {
        // General movement logic
    }
}

public class Robot extends Vehicle {
    @Override
    public void move() {
        // Implement robot-specific movement logic
    }
}
```

2.1.9 Advanced concepts

Method overloading

Method overloading allows a class to have multiple methods with the same name but different parameter lists. This provides flexibility in how methods can be called and allows for different behaviors based on the input parameters.

```java
public class Robot {
    public void move() {
        // Move with default behavior
    }

    public void move(int speed) {
        // Move with specified speed
    }

    public void move(int speed, int direction) {
        // Move with specified speed and direction
```

(continues on next page)

(continued from previous page)

```
    }
}
```

Attention

- Python does not support method overloading in the same way as Java, but you can achieve similar functionality using default parameters or variable-length argument lists.
- In Java, method overloading is determined at compile time based on the method signature, while in Python, you can use `args` and `kwargs` to create flexible methods that can handle different types and numbers of arguments.

Method overriding

Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its superclass. This is a key feature of polymorphism, enabling subclasses to modify or extend the behavior of methods inherited from the superclass.

```python
class Vehicle:
    def move(self):
        print("Vehicle is moving")

class Robot(Vehicle):
    def move(self):
        print("Robot is moving")  # Override the move method
```

```java
public class Vehicle {
    public void move() {
        System.out.println("Vehicle is moving");
    }
}

public class Robot extends Vehicle {
    @Override
    public void move() {
        System.out.println("Robot is moving"); // Override the move method
    }
}
```

Dynamic binding

Dynamic binding, also known as late binding, is a mechanism where the method that gets called is determined at runtime based on the actual type of the object, rather than the type of the reference. This allows for more flexible and dynamic behavior in object-oriented programming.

```python
class Vehicle:
    def move(self):
        print("Vehicle is moving")

class Robot(Vehicle):
    def move(self):
        print("Robot is moving")

def make_vehicle_move(vehicle):
    vehicle.move()  # Dynamic binding occurs here
```

(continues on next page)

(continued from previous page)

```
my_robot = Robot()
make_vehicle_move(my_robot)  # Output: "Robot is moving"
```

```
public class Vehicle {
    public void move() {
        System.out.println("Vehicle is moving");
    }
}

public class Robot extends Vehicle {
    @Override
    public void move() {
        System.out.println("Robot is moving");
    }
}
```

Late binding

Late binding is a related concept to dynamic binding, where the method to be called is determined at runtime. In Java, this is achieved through the use of virtual methods, which are methods that can be overridden in subclasses. In Python, all methods are virtual by default, allowing for late binding without any special syntax.

```
public class Vehicle {
    public void move() {
        System.out.println("Vehicle is moving");
    }
}

public class Robot extends Vehicle {
    @Override
    public void move() {
        System.out.println("Robot is moving");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle myVehicle = new Robot(); // Upcasting to Vehicle
        myVehicle.move(); // Late binding occurs, output: "Robot is moving"
    }
}
```

Attention

- In Python, late binding is inherent to the language, so you can simply call methods on objects without worrying about the binding mechanism.
- In Java, ensure that methods you want to be overridden are not declared as `final`, and use the `@Override` annotation to indicate that a method is intended to override a superclass method for better readability and error checking.

Early binding

Early binding, also known as static binding, is a mechanism where the method that gets called is determined at compile time based on the type of the reference. This is typically used for static methods, private methods, and final methods in Java, which cannot be overridden.

```
public class Vehicle {
    public static void staticMove() {
        System.out.println("Vehicle is moving");
    }

    private void privateMove() {
        System.out.println("Vehicle is moving privately");
    }

    public final void finalMove() {
        System.out.println("Vehicle is moving finally");
    }
}

public class Robot extends Vehicle {
    // Cannot override staticMove, privateMove, or finalMove
}
```

Attention

- In Python, early binding is not a common concept due to the dynamic nature of the language, but you can achieve similar behavior by using static methods or by defining methods that are not intended to be overridden.
- In Java, use static methods for behavior that should not be overridden, and use private methods for internal behavior that should not be accessible outside the class. Use final methods to prevent overriding while still allowing access to the method from subclasses.

Operator overloading

Operator overloading allows you to define custom behavior for standard operators (e.g., +, -, *, /) when they are used with instances of your classes. This can make your classes more intuitive and easier to use.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)
```

```
public class Vector {
    private int x;
    private int y;

    public Vector(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public Vector add(Vector other) {
```

(continues on next page)

(continued from previous page)

```
        return new Vector(this.x + other.x, this.y + other.y);
    }
}
```

Multiple inheritance

Multiple inheritance allows a class to inherit from more than one superclass. This can lead to ambiguity and complexity, especially when the same method or attribute is defined in multiple superclasses, which is known as the "diamond problem". To avoid this issue, some languages like Java do not support multiple inheritance of classes but allow it through interfaces.

In Python, multiple inheritance is supported, but you should be cautious when using it to avoid the diamond problem. Python uses the C3 linearization algorithm to determine the method resolution order (MRO) when multiple inheritance is involved.

```
class A:
    def method(self):
        print("Method in A")

class B(A):
    def method(self):
        print("Method in B")

class C(A):
    def method(self):
        print("Method in C")

class D(B, C):
    pass

d = D()
d.method()  # Output will depend on the MRO, typically "Method in B"
```

Attention

- In Java, use interfaces to achieve multiple inheritance of behavior without the complications of multiple inheritance of state.
- In Python, be mindful of the method resolution order (MRO) when using multiple inheritance, and consider using composition or mixins as alternatives if it leads to complex hierarchies.

Diamond problem

The diamond problem occurs in multiple inheritance when a class inherits from two classes that both inherit from a common superclass. This can lead to ambiguity when the subclass tries to access a method or attribute that is defined in the common superclass, as it may not be clear which version of the method or attribute should be used.

```
class A:
    def method(self):
        print("Method in A")

class B(A):
    def method(self):
        print("Method in B")

class C(A):
```

(continues on next page)

(continued from previous page)

```
    def method(self):
        print("Method in C")

class D(B, C):
    pass

d = D()
d.method()  # Output will depend on the MRO, typically "Method in B"
```

Attention

- In Java, the diamond problem is avoided by not allowing multiple inheritance of classes. Instead, you can use interfaces to achieve similar functionality without the ambiguity.
- In Python, be aware of the method resolution order (MRO) when using multiple inheritance, and consider using composition or mixins as alternatives if it leads to complex hierarchies that may cause the diamond problem.

Delegation

Delegation is a design pattern where an object handles a request by delegating it to another object. This allows for flexible code reuse and can help to avoid issues with multiple inheritance.

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()  # Car has an Engine (delegation)

    def start(self):
        self.engine.start()  # Delegate the start action to the Engine
```

```
public class Engine {
    public void start() {
        System.out.println("Engine started");
    }
}

public class Car {
    private Engine engine; // Car has an Engine (delegation)

    public Car() {
        this.engine = new Engine(); // Initialize the engine
    }

    public void start() {
        this.engine.start(); // Delegate the start action to the Engine
    }
}
```

Inner or netsted classes

Inner or nested classes are classes defined within another class. They can be used to logically group classes that are only used in one place, to increase encapsulation, and to improve readability.

```
public class OuterClass {
    private String outerField;

    public OuterClass(String outerField) {
        this.outerField = outerField;
    }

    // Inner class
    public class InnerClass {
        public void display() {
            System.out.println("Outer field: " + outerField); // Accessing outer␣
↪class field
        }
    }
}
```

In Python, you can also define nested classes, but they are not commonly used and do not have the same level of access to the outer class's attributes as Java's inner classes.

```
class OuterClass:
    def __init__(self, outer_field):
        self.outer_field = outer_field

    class InnerClass:
        def display(self, outer_instance):
            print(f"Outer field: {outer_instance.outer_field}")  # Accessing outer␣
↪class field
```

2.1.10 Abstraction mechanisms

Abstract classes

Abstract classes are classes that cannot be instantiated on their own and are meant to be subclassed. They can contain abstract methods, which are declared without an implementation and must be implemented by subclasses. This allows for defining a common interface for a group of related classes while allowing each subclass to provide its own specific implementation.

```
public abstract class Shape {
    public abstract double area(); // Abstract method to be implemented by subclasses
}

public class Circle extends Shape {
    private double radius;
    public Circle(double radius) {
        this.radius = radius;
    }
    @Override
    public double area() {
        return Math.PI * radius * radius; // Implementation of the abstract method
    }
}

public class Rectangle extends Shape {
```

(continues on next page)

(continued from previous page)

```
    private double width;
    private double height;
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
    @Override
    public double area() {
        return width * height; // Implementation of the abstract method
    }
}
```

In Python, you can achieve similar functionality using the `abc` module to create abstract base classes and abstract methods.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass  # Abstract method to be implemented by subclasses

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius  # Implementation of the abstract␣
↪method

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height  # Implementation of the abstract method
```

Interfaces

An interface is a contract that defines a set of methods that a class must implement. It allows for defining a common interface for a group of related classes while allowing each class to provide its own specific implementation. In Java, interfaces are defined using the `interface` keyword, and classes implement interfaces using the `implements` keyword.

```
public interface Drawable {
    void draw(); // Interface method
}

public class Circle implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a circle"); // Implementation of the interface␣
↪method
    }
}
```

(continues on next page)

(continued from previous page)

```
public class Rectangle implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a rectangle"); // Implementation of the interface␣
↪method
    }
}
```

In Python, you can achieve a similar effect using abstract base classes and the abc module to define interfaces and their implementations.

```
from abc import ABC, abstractmethod

class Drawable(ABC):
    @abstractmethod
    def draw(self):
        pass  # Interface method

class Circle(Drawable):
    def draw(self):
        print("Drawing a circle")  # Implementation of the interface method

class Rectangle(Drawable):
    def draw(self):
        print("Drawing a rectangle")  # Implementation of the interface method
```

Marker interface

A marker interface is an interface that does not contain any methods or fields and is used to indicate that a class has a certain property or should be treated in a specific way. In Java, marker interfaces are often used to indicate that a class can be serialized, cloned, or is thread-safe.

```
public interface Serializable {
    // No methods, just a marker interface
}

public class MyClass implements Serializable {
    // This class can be serialized because it implements the Serializable marker␣
↪interface
}
```

In Python, there is no direct equivalent to marker interfaces, but you can achieve similar functionality using class decorators or by defining a base class that serves as a marker.

```
class Serializable:
    pass  # Marker class

@Serializable
class MyClass:
    # This class can be treated as serializable because it is decorated with the␣
↪Serializable marker
    pass
```

Functional interface

A functional interface is an interface that contains exactly one abstract method. It is used to represent a single function or behavior that can be passed around as an object. In Java, functional interfaces are often used in conjunction with lambda expressions to provide a concise way to implement the interface.

```
@FunctionalInterface
public interface Runnable {
    void run(); // Single abstract method
}

public class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Running a task"); // Implementation of the functional␣
↪interface method
    }
}
```

In Python, you can achieve a similar effect using callable classes or by using the `functools` module to create function objects.

```
class Runnable:
    def __call__(self):
        print("Running a task")  # Implementation of the functional interface method

my_runnable = Runnable()
my_runnable()  # Output: "Running a task"
```

2.1.11 Object-oriented design concepts

SOLID principles

SOLID is an acronym for five design principles that help to create maintainable and scalable software. These principles are:

- **Single Responsibility Principle:** A class should have only one reason to change, meaning it should have only one responsibility or job.
- **Open-Closed Principle:** Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification, meaning you should be able to add new functionality without changing existing code.
- **Liskov Substitution Principle:** Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program, meaning that subclasses should be able to stand in for their superclasses without causing errors or unexpected behavior.
- **Interface Segregation Principle:** Clients should not be forced to depend on interfaces they do not use, meaning that interfaces should be specific to the needs of the clients that use them, rather than being general-purpose interfaces that may contain methods that are not relevant to all clients.
- **Dependency Inversion Principle:** High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions. This means that you should depend on interfaces or abstract classes rather than concrete implementations, which allows for more flexible and decoupled code.

Single responsibility principle

Single responsibility principle states that a class should have only one reason to change, meaning it should have only one responsibility or job. This principle helps to create classes that are focused and easier to maintain, as changes to one responsibility will not affect other responsibilities. By adhering to this principle, you can create a

more modular and cohesive codebase, where each class has a clear purpose and is easier to understand and modify when necessary.

```python
class Robot:
    def __init__(self, name):
        self.name = name

    def move(self):
        # Code for moving the robot
        pass

    def communicate(self):
        # Code for communicating with other robots
        pass
```

```java
public class Robot {
    private String name;

    public Robot(String name) {
        this.name = name;
    }

    public void move() {
        // Code for moving the robot
    }

    public void communicate() {
        // Code for communicating with other robots
    }
}
```

Open-closed principle

Open-closed principle states that software entities (classes, modules, functions, etc.) should be open for extension but closed for modification. This means that you should be able to add new functionality to a class without changing its existing code. By adhering to this principle, you can create a more flexible and maintainable codebase, as new features can be added without risking the introduction of bugs or breaking existing functionality. This can be achieved through the use of inheritance, interfaces, and composition, allowing you to extend the behavior of existing classes without modifying their code.

```python
class Robot:
    def __init__(self, name):
        self.name = name

    def move(self):
        # Code for moving the robot
        pass

class AdvancedRobot(Robot):
    def communicate(self):
        # Code for communicating with other robots
        pass
```

```java
public class Robot {
    private String name;

    public Robot(String name) {
```

(continues on next page)

(continued from previous page)

```
        this.name = name;
    }

    public void move() {
        // Code for moving the robot
    }
}

public class AdvancedRobot extends Robot {
    public AdvancedRobot(String name) {
        super(name);
    }

    public void communicate() {
        // Code for communicating with other robots
    }
}
```

Liskov substitution principle

Liskov substitution principle states that objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program. This means that subclasses should be able to stand in for their superclasses without causing errors or unexpected behavior. By adhering to this principle, you can create a more robust and flexible codebase, as subclasses can be used interchangeably with their superclasses, allowing for greater code reuse and easier maintenance. This can be achieved by ensuring that subclasses do not violate the expectations set by the superclass, such as maintaining the same method signatures, adhering to the same contracts, and not introducing side effects that would break the functionality of the superclass.

```
class Vehicle:
    def move(self):
        pass

class Robot(Vehicle):
    def move(self):
        # Implement robot-specific movement logic
        pass

def make_vehicle_move(vehicle):
    vehicle.move()  # This should work for both Vehicle and Robot instances
```

```
public class Vehicle {
    public void move() {
        // General movement logic
    }
}

public class Robot extends Vehicle {
    @Override
    public void move() {
        // Implement robot-specific movement logic
    }
}
```

Interface segregation principle

Interface segregation principle states that clients should not be forced to depend on interfaces they do not use. This means that interfaces should be specific to the needs of the clients that use them, rather than being general-purpose interfaces that may contain methods that are not relevant to all clients. By adhering to this principle, you can create a more modular and maintainable codebase, as clients will only depend on the functionality they actually need, reducing coupling and increasing cohesion. This can be achieved by breaking down large interfaces into smaller, more specific interfaces that cater to the needs of different clients.

```java
public interface Movable {
    void move(); // Interface for movement
}

public interface Communicable {
    void communicate(); // Interface for communication
}

public class Robot implements Movable, Communicable {
    @Override
    public void move() {
        // Implement movement logic
    }

    @Override
    public void communicate() {
        // Implement communication logic
    }
}
```

In Python, you can achieve a similar effect using abstract base classes and the `abc` module to define specific interfaces for different functionalities.

```python
from abc import ABC, abstractmethod

class Movable(ABC):
    @abstractmethod
    def move(self):
        pass  # Interface for movement

class Communicable(ABC):
    @abstractmethod
    def communicate(self):
        pass  # Interface for communication

class Robot(Movable, Communicable):
    def move(self):
        # Implement movement logic
        pass

    def communicate(self):
        # Implement communication logic
        pass
```

GRASP principles

GRASP (General Responsibility Assignment Software Patterns) is a set of principles for assigning responsibilities to classes and objects in object-oriented design. These principles help to create a well-structured and maintainable codebase by guiding the design of classes and their interactions. The GRASP principles include:

- **Information Expert:** Assign responsibility to the class that has the necessary information to fulfill it.
- **Creator:** Assign responsibility to the class that creates instances of other classes.
- **Controller:** Assign responsibility to a class that represents the overall system or a use case scenario
- **Low Coupling:** Assign responsibilities in a way that minimizes the dependencies between classes.
- **High Cohesion:** Assign responsibilities in a way that keeps related behavior together and promotes a clear and focused design.
- **Polymorphism:** Use polymorphic behavior to handle variations in behavior based on the type of object.
- **Pure Fabrication:** Assign responsibilities to a class that does not represent a concept in the problem domain to achieve low coupling and high cohesion.
- **Indirection:** Assign responsibility to an intermediate class to mediate between other classes or to decouple them.
- **Protected Variations:** Assign responsibilities in a way that protects elements from variations in other elements, such as using interfaces or abstract classes to shield clients from changes in the implementation.

Information expert

Information expert is a principle that suggests assigning responsibility to the class that has the necessary information to fulfill it. This means that the class that has the most relevant data or knowledge about a particular responsibility should be responsible for handling that responsibility. By following this principle, you can create a more cohesive and maintainable codebase, as responsibilities are assigned to classes that are best suited to handle them, reducing the need for unnecessary dependencies and improving the overall design of the system.

```python
class Robot:
    def __init__(self, name, speed):
        self.name = name
        self.speed = speed  # Robot has the necessary information about its speed

    def move(self):
        # Use the speed information to move the robot
        pass
```

```java
public class Robot {
    private String name;
    private int speed; // Robot has the necessary information about its speed

    public Robot(String name, int speed) {
        this.name = name;
        this.speed = speed;
    }

    public void move() {
        // Use the speed information to move the robot
    }
}
```

Creator

Creator is a principle that suggests assigning responsibility to the class that creates instances of other classes. This means that if a class is responsible for creating an instance of another class, it should also be responsible for managing the lifecycle of that instance. By following this principle, you can create a more cohesive and maintainable codebase, as the responsibility for creating and managing objects is centralized in one place, reducing the need for unnecessary dependencies and improving the overall design of the system.

```python
class Sensor:
    def __init__(self, sensor_id):
        self.sensor_id = sensor_id

class Robot:
    def __init__(self, name):
        self.name = name
        self.sensor = Sensor(1)  # Robot creates and manages the Sensor instance
```

```java
public class Sensor {
    private int sensorId;
    public Sensor(int sensorId) {
        this.sensorId = sensorId;
    }
}

public class Robot {
    private String name;
    private Sensor sensor; // Robot creates and manages the Sensor instance

    public Robot(String name) {
        this.name = name;
        this.sensor = new Sensor(1); // Initialize the sensor
    }
}
```

Controller

Controller is a principle that suggests assigning responsibility to a class that represents the overall system or a use case scenario. This means that a controller class should be responsible for handling the flow of control and coordinating the interactions between other classes in the system. By following this principle, you can create a more cohesive and maintainable codebase, as the controller class can serve as a central point of control for the system, reducing the need for unnecessary dependencies and improving the overall design of the system.

```python
class RobotController:
    def __init__(self, robot):
        self.robot = robot

    def execute_move(self):
        # Coordinate the movement of the robot
        self.robot.move()
```

```java
public class RobotController {
    private Robot robot;

    public RobotController(Robot robot) {
        this.robot = robot;
    }

    public void executeMove() {
        // Coordinate the movement of the robot
        this.robot.move();
    }
}
```

Low coupling

Low coupling is a principle that suggests assigning responsibilities in a way that minimizes the dependencies between classes. This means that classes should be designed to be as independent as possible, with minimal knowledge of each other's internal workings. By following this principle, you can create a more modular and maintainable codebase, as changes to one class will have minimal impact on other classes, reducing the risk of introducing bugs and making it easier to modify and extend the system in the future.

```
class Sensor:
    def read_value(self):
        pass

class Robot:
    def __init__(self, sensor):
        self.sensor = sensor  # Robot depends on Sensor, but Sensor does not depend␣
↪on Robot

    def move(self):
        value = self.sensor.read_value()  # Robot uses Sensor, but Sensor is not␣
↪tightly coupled to Robot
```

```
public class Sensor {
    public int readValue() {
        // Read sensor value
        return 0;
    }
}

public class Robot {
    private Sensor sensor; // Robot depends on Sensor, but Sensor does not depend on␣
↪Robot

    public Robot(Sensor sensor) {
        this.sensor = sensor; // Initialize the sensor
    }

    public void move() {
        int value = this.sensor.readValue(); // Robot uses Sensor, but Sensor is not␣
↪tightly coupled to Robot
    }
}
```

High cohesion

High cohesion is a principle that suggests assigning responsibilities in a way that keeps related behavior together and promotes a clear and focused design. This means that classes should be designed to have a single, well-defined purpose, with all of their methods and attributes related to that purpose. By following this principle, you can create a more maintainable and understandable codebase, as classes with high cohesion are easier to understand, modify, and extend without affecting unrelated functionality. This can be achieved by grouping related methods and attributes together in the same class, and by avoiding the inclusion of unrelated functionality in a single class.

```
class Robot:
    def __init__(self, name):
        self.name = name

    def move(self):
        # Code for moving the robot
```

(continues on next page)

(continued from previous page)

```
        pass

    def communicate(self):
        # Code for communicating with other robots
        pass
```

```
public class Robot {
    private String name;

    public Robot(String name) {
        this.name = name;
    }

    public void move() {
        // Code for moving the robot
    }

    public void communicate() {
        // Code for communicating with other robots
    }
}
```

Polymorphism

Polymorphism is a principle that allows objects of different classes to be treated as objects of a common superclass. This means that you can write code that works with objects of different types as long as they share a common interface or superclass. By following this principle, you can create a more flexible and maintainable codebase, as you can write code that is not dependent on specific implementations, allowing for greater code reuse and easier maintenance. This can be achieved through the use of inheritance, interfaces, and abstract classes, allowing you to define common behavior in a superclass or interface and have different subclasses provide their own specific implementations.

```
class Vehicle:
    def move(self):
        pass

class Robot(Vehicle):
    def move(self):
        # Implement robot-specific movement logic
        pass

def make_vehicle_move(vehicle):
    vehicle.move()  # This can work with any subclass of Vehicle, demonstrating␣
↪polymorphism
```

```
public class Vehicle {
    public void move() {
        // General movement logic
    }
}

public class Robot extends Vehicle {
    @Override
    public void move() {
        // Implement robot-specific movement logic
```

(continues on next page)

(continued from previous page)

```
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle myVehicle = new Robot(); // Upcasting to Vehicle
        myVehicle.move(); // This can work with any subclass of Vehicle,␣
↪demonstrating polymorphism
    }
}
```

Pure fabrication

Pure fabrication is a principle that suggests assigning responsibilities to a class that does not represent a concept in the problem domain to achieve low coupling and high cohesion. This means that you can create a class that serves as a helper or utility class to handle responsibilities that do not fit well within the existing domain classes, allowing you to keep your domain classes focused and cohesive while still providing the necessary functionality. By following this principle, you can create a more maintainable and flexible codebase, as the pure fabrication class can be modified or extended without affecting the core domain classes, reducing the risk of introducing bugs and making it easier to adapt to changing requirements.

```
class SensorDataProcessor:
    @staticmethod
    def process_data(data):
        # Code for processing sensor data that does not fit well within the Robot␣
↪class
        pass

class Robot:
    def __init__(self, name):
        self.name = name

    def move(self):
        # Code for moving the robot
        pass

    def communicate(self):
        # Code for communicating with other robots
        pass
```

```
public class SensorDataProcessor {
    public static void processData(String data) {
        // Code for processing sensor data that does not fit well within the Robot␣
↪class
    }
}

public class Robot {
    private String name;

    public Robot(String name) {
        this.name = name;
    }

    public void move() {
        // Code for moving the robot
```

(continues on next page)

(continued from previous page)

```
    }

    public void communicate() {
        // Code for communicating with other robots
    }
}
```

Indirection

Indirection is a principle that suggests assigning responsibility to an intermediate class to mediate between other classes or to decouple them. This means that you can create a class that serves as an intermediary between two or more classes, allowing them to interact without being directly dependent on each other. By following this principle, you can create a more flexible and maintainable codebase, as the indirection class can be modified or extended without affecting the classes it mediates between, reducing the risk of introducing bugs and making it easier to adapt to changing requirements. This can be achieved through the use of design patterns such as the mediator pattern, which provides a central point of control for interactions between classes, or the adapter pattern, which allows classes with incompatible interfaces to work together.

```
class Mediator:
    def __init__(self):
        self.robot = None
        self.sensor = None

    def set_robot(self, robot):
        self.robot = robot

    def set_sensor(self, sensor):
        self.sensor = sensor

    def coordinate(self):
        # Code to coordinate interactions between the robot and sensor
        pass

class Robot:
    def move(self):
        pass

class Sensor:
    def read_value(self):
        pass
```

```
public class Mediator {
    private Robot robot;
    private Sensor sensor;

    public void setRobot(Robot robot) {
        this.robot = robot;
    }

    public void setSensor(Sensor sensor) {
        this.sensor = sensor;
    }

    public void coordinate() {
        // Code to coordinate interactions between the robot and sensor
    }
```

(continues on next page)

(continued from previous page)

```
}

public class Robot {
    public void move() {
        // Code for moving the robot
    }
}

public class Sensor {
    public int readValue() {
        // Code for reading sensor value
        return 0;
    }
}
```

Protected variations

Protected variations is a principle that suggests assigning responsibilities in a way that protects elements from variations in other elements, such as using interfaces or abstract classes to shield clients from changes in the implementation. This means that you can design your classes and their interactions in a way that allows for changes in one part of the system without affecting other parts, reducing the risk of introducing bugs and making it easier to adapt to changing requirements. By following this principle, you can create a more flexible and maintainable codebase, as clients will be insulated from changes in the implementation, allowing for greater code reuse and easier maintenance. This can be achieved through the use of design patterns such as the adapter pattern, which allows classes with incompatible interfaces to work together, or the facade pattern, which provides a simplified interface to a complex subsystem.

```
class SensorInterface:
    def read_value(self):
        pass  # Interface to shield clients from changes in the sensor implementation

class Sensor(SensorInterface):
    def read_value(self):
        # Code for reading sensor value
        return 0

class Robot:
    def __init__(self, sensor: SensorInterface):
        self.sensor = sensor  # Robot depends on the SensorInterface, not the␣
↪concrete Sensor implementation

    def move(self):
        value = self.sensor.read_value()  # Robot can work with any implementation of␣
↪SensorInterface
```

```
public interface SensorInterface {
    int readValue(); // Interface to shield clients from changes in the sensor␣
↪implementation
}

public class Sensor implements SensorInterface {
    @Override
    public int readValue() {
        // Code for reading sensor value
        return 0;
    }
```

(continues on next page)

(continued from previous page)

```
}

public class Robot {
    private SensorInterface sensor; // Robot depends on the SensorInterface, not the␣
↪concrete Sensor implementation

    public Robot(SensorInterface sensor) {
        this.sensor = sensor; // Initialize the sensor
    }

    public void move() {
        int value = this.sensor.readValue(); // Robot can work with any␣
↪implementation of SensorInterface
    }
}
```

Separation of concerns

Separation of concerns is a principle that suggests dividing a software system into distinct sections, each of which addresses a separate concern or aspect of the system. This means that different parts of the system should be responsible for different functionalities, allowing for greater modularity and maintainability. By following this principle, you can create a more organized and understandable codebase, as each section of the code will have a clear purpose and will be easier to modify and extend without affecting other parts of the system. This can be achieved through the use of design patterns such as the model-view-controller (MVC) pattern, which separates the data model, user interface, and control logic into distinct components.

```
class Model:
    def __init__(self):
        self.data = "Model data"

class View:
    def display(self, data):
        print(f"Displaying: {data}")

class Controller:
    def __init__(self, model, view):
        self.model = model
        self.view = view

    def update_view(self):
        self.view.display(self.model.data)

# Example usage
model = Model()
view = View()
controller = Controller(model, view)
controller.update_view()  # Output: "Displaying: Model data"
```

```
public class Model {
    private String data = "Model data";

    public String getData() {
        return data;
    }
}
```

(continues on next page)

(continued from previous page)

```
public class View {
    public void display(String data) {
        System.out.println("Displaying: " + data);
    }
}

public class Controller {
    private Model model;
    private View view;

    public Controller(Model model, View view) {
        this.model = model;
        this.view = view;
    }

    public void updateView() {
        view.display(model.getData());
    }
}

// Example usage
public class Main {
    public static void main(String[] args) {
        Model model = new Model();
        View view = new View();
        Controller controller = new Controller(model, view);
        controller.updateView(); // Output: "Displaying: Model data"
    }
}
```

Coupling and cohesion

Coupling refers to the degree of interdependence between software modules, while cohesion refers to the degree to which the elements within a module belong together. High coupling means that modules are highly dependent on each other, while low coupling means that modules are independent and can be modified without affecting other modules. High cohesion means that the elements within a module are closely related and work together to achieve a specific purpose, while low cohesion means that the elements within a module are unrelated and do not work together effectively. By striving for low coupling and high cohesion in your software design, you can create a more maintainable and flexible codebase, as changes to one module will have minimal impact on other modules, and each module will have a clear and focused purpose.

```
class Sensor:
    def read_value(self):
        pass  # Low coupling, as Sensor does not depend on other modules

class Robot:
    def __init__(self, sensor):
        self.sensor = sensor  # Low coupling, as Robot depends on Sensor but Sensor␣
↪does not depend on Robot

    def move(self):
        value = self.sensor.read_value()  # High cohesion, as move method is focused␣
↪on robot movement logic
```

```
public class Sensor {
    public int readValue() {
```

(continues on next page)

(continued from previous page)

```
        // Low coupling, as Sensor does not depend on other modules
        return 0;
    }
}

public class Robot {
    private Sensor sensor; // Low coupling, as Robot depends on Sensor but Sensor␣
↪does not depend on Robot

    public Robot(Sensor sensor) {
        this.sensor = sensor; // Initialize the sensor
    }

    public void move() {
        int value = this.sensor.readValue(); // High cohesion, as move method is␣
↪focused on robot movement logic
    }
}
```

Law of Demeter

Demeter is a principle that suggests that a module should not know about the internal details of other modules, and should only interact with its immediate collaborators. This means that a module should only call methods on objects that it directly owns or has a direct relationship with, rather than reaching through multiple layers of objects to access functionality. By following this principle, you can create a more maintainable and flexible codebase, as modules will be less dependent on the internal structure of other modules, reducing the risk of introducing bugs and making it easier to modify and extend the system in the future. This can be achieved by designing your classes and their interactions in a way that minimizes the need for modules to access the internal details of other modules, such as by using interfaces or abstract classes to define clear boundaries between modules.

Law of demeter states that a module should not know about the internal details of other modules, and should only interact with its immediate collaborators. This means that a module should only call methods on objects that it directly owns or has a direct relationship with, rather than reaching through multiple layers of objects to access functionality. By following this principle, you can create a more maintainable and flexible codebase, as modules will be less dependent on the internal structure of other modules, reducing the risk of introducing bugs and making it easier to modify and extend the system in the future. This can be achieved by designing your classes and their interactions in a way that minimizes the need for modules to access the internal details of other modules, such as by using interfaces or abstract classes to define clear boundaries between modules.

```
class Sensor:
    def read_value(self):
        pass

class Robot:
    def __init__(self, sensor):
        self.sensor = sensor  # Robot has a direct relationship with Sensor

    def move(self):
        value = self.sensor.read_value()  # Robot interacts with its immediate␣
↪collaborator, Sensor
```

```
public class Sensor {
    public int readValue() {
        // Code for reading sensor value
        return 0;
    }
```

(continues on next page)

(continued from previous page)

```
}

public class Robot {
    private Sensor sensor; // Robot has a direct relationship with Sensor

    public Robot(Sensor sensor) {
        this.sensor = sensor; // Initialize the sensor
    }

    public void move() {
        int value = this.sensor.readValue(); // Robot interacts with its immediate␣
↪collaborator, Sensor
    }
}
```

2.1.12 Design patterns

Design patterns are reusable solutions to common software design problems that have been proven effective over time. They provide a way to structure your code in a way that is maintainable, flexible, and scalable. Design patterns can be categorized into three main types: creational patterns, structural patterns, and behavioral patterns.

- **Creational patterns:** focus on object creation mechanisms, structural patterns focus on the composition of classes and objects.
- **Behavioral patterns:** focus on communication between objects. By using design patterns in your software design, you can create a more robust and maintainable codebase, as they provide a common language and structure for solving common design problems, making it easier to understand and modify the code in the future.
- **Structural patterns:** focus on the composition of classes and objects, while behavioral patterns focus on communication between objects. By using design patterns in your software design, you can create a more robust and maintainable codebase, as they provide a common language and structure for solving common design problems, making it easier to understand and modify the code in the future.

Creational patterns

Creational pattern is a design pattern that focuses on object creation mechanisms, trying to create objects in a manner suitable to the situation. The basic form of object creation could lead to design problems or added complexity in the design. Creational design patterns solve this problem by somehow controlling this object creation. Some common creational patterns include:

- **Singleton:** Ensures that a class has only one instance and provides a global point of access to it.
- **Factory method:** Defines an interface for creating an object, but lets subclasses alter the type of objects that will be created.
- **Abstract factory:** Provides an interface for creating families of related or dependent objects without specifying their concrete classes.
- **Builder:** Separates the construction of a complex object from its representation, allowing the same construction process to create different representations.
- **Prototype:** Specifies the kinds of objects to create using a prototypical instance, and creates new objects by copying this prototype.

Singleton

Singleton is a design pattern that ensures a class has only one instance and provides a global point of access to it. This is useful when you want to control access to a shared resource, such as a database connection or a configuration manager, and ensure that there is only one instance of that resource throughout the application. The

singleton pattern can be implemented in various programming languages, and it typically involves making the constructor private and providing a static method to get the instance of the class.

```python
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(Singleton, cls).__new__(cls)
        return cls._instance
```

```java
public class Singleton {
    private static Singleton instance;

    private Singleton() {
        // Private constructor to prevent instantiation
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Factory method

Factory method is a design pattern that defines an interface for creating an object, but lets subclasses alter the type of objects that will be created. This pattern allows a class to defer instantiation to subclasses, providing a way to create objects without specifying the exact class of object that will be created. By using the factory method pattern, you can create a more flexible and maintainable codebase, as it allows you to add new types of objects without modifying existing code, adhering to the open-closed principle. This can be achieved by defining a factory method in a base class or interface, and then implementing that method in subclasses to create specific types of objects.

```python
class VehicleFactory:
    def create_vehicle(self):
        pass  # Factory method to be implemented by subclasses

class CarFactory(VehicleFactory):
    def create_vehicle(self):
        return Car()  # Create a Car object

class BikeFactory(VehicleFactory):
    def create_vehicle(self):
        return Bike()  # Create a Bike object
```

```java
public abstract class VehicleFactory {
    public abstract Vehicle createVehicle(); // Factory method to be implemented by␣
↪subclasses
}

public class CarFactory extends VehicleFactory {
    @Override
    public Vehicle createVehicle() {
        return new Car(); // Create a Car object
```

(continues on next page)

(continued from previous page)

```
    }
}

public class BikeFactory extends VehicleFactory {
    @Override
    public Vehicle createVehicle() {
        return new Bike(); // Create a Bike object
    }
}
```

Abstract factory

Abstract factory is a design pattern that provides an interface for creating families of related or dependent objects without specifying their concrete classes. This pattern allows you to create a suite of related products without being tied to specific implementations, making it easier to maintain and extend your codebase. By using the abstract factory pattern, you can create a more flexible and maintainable codebase, as it allows you to add new families of products without modifying existing code, adhering to the open-closed principle. This can be achieved by defining an abstract factory interface that declares methods for creating each type of product, and then implementing that interface in concrete factory classes that create specific families of products.

```
class AbstractFactory:
    def create_product_a(self):
        pass  # Method to create product A

    def create_product_b(self):
        pass  # Method to create product B

class ConcreteFactory1(AbstractFactory):
    def create_product_a(self):
        return ProductA1()  # Create product A1

    def create_product_b(self):
        return ProductB1()  # Create product B1

class ConcreteFactory2(AbstractFactory):
    def create_product_a(self):
        return ProductA2()  # Create product A2

    def create_product_b(self):
        return ProductB2()  # Create product B2
```

```
public abstract class AbstractFactory {
    public abstract ProductA createProductA(); // Method to create product A
    public abstract ProductB createProductB(); // Method to create product B
}

public class ConcreteFactory1 extends AbstractFactory {
    @Override
    public ProductA createProductA() {
        return new ProductA1(); // Create product A1
    }

    @Override
    public ProductB createProductB() {
        return new ProductB1(); // Create product B1
    }
```

(continues on next page)

(continued from previous page)

```
}

public class ConcreteFactory2 extends AbstractFactory {
    @Override
    public ProductA createProductA() {
        return new ProductA2(); // Create product A2
    }

    @Override
    public ProductB createProductB() {
        return new ProductB2(); // Create product B2
    }
}
```

Builder

Builder is a design pattern that separates the construction of a complex object from its representation, allowing the same construction process to create different representations. This pattern is useful when you want to create an object that requires multiple steps to construct, and you want to be able to create different variations of that object without changing the construction process. By using the builder pattern, you can create a more flexible and maintainable codebase, as it allows you to add new variations of the object without modifying existing code, adhering to the open-closed principle. This can be achieved by defining a builder interface that declares methods for constructing each part of the object, and then implementing that interface in concrete builder classes that create specific variations of the object.

```
class Builder:
    def build_part_a(self):
        pass  # Method to build part A

    def build_part_b(self):
        pass  # Method to build part B

    def get_result(self):
        pass  # Method to return the final product

class ConcreteBuilder1(Builder):
    def build_part_a(self):
        # Build part A for product 1
        pass

    def build_part_b(self):
        # Build part B for product 1
        pass

    def get_result(self):
        return Product1()  # Return the final product 1

class ConcreteBuilder2(Builder):
    def build_part_a(self):
        # Build part A for product 2
        pass

    def build_part_b(self):
        # Build part B for product 2
        pass
```

(continues on next page)

(continued from previous page)

```
    def get_result(self):
        return Product2()  # Return the final product 2
```

```
public abstract class Builder {
    public abstract void buildPartA(); // Method to build part A
    public abstract void buildPartB(); // Method to build part B
    public abstract Product getResult(); // Method to return the final product
}

public class ConcreteBuilder1 extends Builder {
    @Override
    public void buildPartA() {
        // Build part A for product 1
    }

    @Override
    public void buildPartB() {
        // Build part B for product 1
    }

    @Override
    public Product getResult() {
        return new Product1(); // Return the final product 1
    }
}

public class ConcreteBuilder2 extends Builder {
    @Override
    public void buildPartA() {
        // Build part A for product 2
    }

    @Override
    public void buildPartB() {
        // Build part B for product 2
    }

    @Override
    public Product getResult() {
        return new Product2(); // Return the final product 2
    }
}
```

Prototype

Prototype is a design pattern that specifies the kinds of objects to create using a prototypical instance, and creates new objects by copying this prototype. This pattern is useful when you want to create new objects that are similar to existing objects, and you want to avoid the overhead of creating new instances from scratch. By using the prototype pattern, you can create a more flexible and maintainable codebase, as it allows you to create new variations of an object without modifying existing code, adhering to the open-closed principle. This can be achieved by defining a prototype interface that declares a method for cloning the object, and then implementing that interface in concrete prototype classes that create specific variations of the object.

```
class Prototype:
    def clone(self):
        pass  # Method to clone the object
```

(continues on next page)

(continued from previous page)

```
class ConcretePrototype(Prototype):
    def clone(self):
        # Code to create a copy of the object
        return ConcretePrototype()  # Return the cloned object
```

```
public abstract class Prototype {
    public abstract Prototype clone(); // Method to clone the object
}

public class ConcretePrototype extends Prototype {
    @Override
    public Prototype clone() {
        // Code to create a copy of the object
        return new ConcretePrototype(); // Return the cloned object
    }
}
```

Structural patterns

Structural patterns are design patterns that focus on the composition of classes and objects, providing ways to create relationships between them to form larger structures. These patterns help to ensure that if one part of a system changes, the entire system doesn't need to change as well. By using structural patterns in your software design, you can create a more flexible and maintainable codebase, as they provide a common language and structure for organizing classes and objects, making it easier to understand and modify the code in the future. Some common structural patterns include:

- **Adapter:** Allows classes with incompatible interfaces to work together by converting the interface of one class into an interface expected by the clients.
- **Decorator:** Allows behavior to be added to individual objects, either statically or dynamically, without affecting the behavior of other objects from the same class.
- **Facade:** Provides a simplified interface to a complex subsystem, making it easier to use and understand.
- **Proxy:** Provides a surrogate or placeholder for another object to control access to it.
- **Composite:** Composes objects into tree structures to represent part-whole hierarchies, allowing clients to treat individual objects and compositions of objects uniformly.

Adapter

Adapter is a design pattern that allows classes with incompatible interfaces to work together by converting the interface of one class into an interface expected by the clients. This pattern is useful when you want to use an existing class that does not have the desired interface, and you want to avoid modifying the existing class or creating a new class that inherits from it. By using the adapter pattern, you can create a more flexible and maintainable codebase, as it allows you to integrate existing classes into your system without modifying them, adhering to the open-closed principle. This can be achieved by defining an adapter class that implements the desired interface and holds a reference to an instance of the existing class, forwarding calls to the existing class as needed.

```
class TargetInterface:
    def request(self):
        pass  # Desired interface for clients

class Adaptee:
    def specific_request(self):
        pass  # Existing class with an incompatible interface
```

(continues on next page)

(continued from previous page)

```
class Adapter(TargetInterface):
    def __init__(self, adaptee):
        self.adaptee = adaptee  # Hold a reference to the existing class

    def request(self):
        return self.adaptee.specific_request()  # Forward calls to the existing class
```

```
public interface TargetInterface {
    void request(); // Desired interface for clients
}

public class Adaptee {
    public void specificRequest() {
        // Existing class with an incompatible interface
    }
}

public class Adapter implements TargetInterface {
    private Adaptee adaptee; // Hold a reference to the existing class

    public Adapter(Adaptee adaptee) {
        this.adaptee = adaptee;
    }

    @Override
    public void request() {
        adaptee.specificRequest(); // Forward calls to the existing class
    }
}
```

Decorator

Decorator is a design pattern that allows behavior to be added to individual objects, either statically or dynamically, without affecting the behavior of other objects from the same class. This pattern is useful when you want to add responsibilities to objects without modifying their class, and you want to be able to add or remove responsibilities at runtime. By using the decorator pattern, you can create a more flexible and maintainable codebase, as it allows you to extend the functionality of objects without modifying existing code, adhering to the open-closed principle. This can be achieved by defining a decorator class that implements the same interface as the objects it decorates and holds a reference to an instance of the object being decorated, forwarding calls to the decorated object and adding additional behavior as needed.

```
class Component:
    def operation(self):
        pass  # Interface for objects that can have responsibilities added to them

class ConcreteComponent(Component):
    def operation(self):
        # Code for the core functionality of the component
        pass

class Decorator(Component):
    def __init__(self, component):
        self.component = component  # Hold a reference to the object being decorated

    def operation(self):
        self.component.operation()  # Forward calls to the decorated object
```

(continues on next page)

(continued from previous page)

```
        # Add additional behavior here
```

```
public interface Component {
    void operation(); // Interface for objects that can have responsibilities added␣
↪to them
}

public class ConcreteComponent implements Component {
    @Override
    public void operation() {
        // Code for the core functionality of the component
    }
}

public class Decorator implements Component {
    private Component component; // Hold a reference to the object being decorated

    public Decorator(Component component) {
        this.component = component;
    }

    @Override
    public void operation() {
        component.operation(); // Forward calls to the decorated object
        // Add additional behavior here
    }
}
```

Facade

Facade is a design pattern that provides a simplified interface to a complex subsystem, making it easier to use and understand. This pattern is useful when you want to hide the complexities of a subsystem from clients and provide a simple interface for them to interact with. By using the facade pattern, you can create a more maintainable and flexible codebase, as it allows you to change the underlying implementation of the subsystem without affecting clients, adhering to the open-closed principle. This can be achieved by defining a facade class that provides methods for clients to interact with the subsystem, and internally uses instances of the subsystem classes to perform the necessary operations.

```
class SubsystemA:
    def operation_a(self):
        pass  # Code for subsystem A

class SubsystemB:
    def operation_b(self):
        pass  # Code for subsystem B

class Facade:
    def __init__(self):
        self.subsystem_a = SubsystemA()
        self.subsystem_b = SubsystemB()

    def simplified_operation(self):
        self.subsystem_a.operation_a()  # Use subsystem A
        self.subsystem_b.operation_b()  # Use subsystem B
        # Provide a simplified interface for clients
```

```
public class SubsystemA {
    public void operationA() {
        // Code for subsystem A
    }
}

public class SubsystemB {
    public void operationB() {
        // Code for subsystem B
    }
}

public class Facade {
    private SubsystemA subsystemA;
    private SubsystemB subsystemB;

    public Facade() {
        this.subsystemA = new SubsystemA();
        this.subsystemB = new SubsystemB();
    }

    public void simplifiedOperation() {
        subsystemA.operationA(); // Use subsystem A
        subsystemB.operationB(); // Use subsystem B
        // Provide a simplified interface for clients
    }
}
```

Proxy

Proxy is a design pattern that provides a surrogate or placeholder for another object to control access to it. This pattern is useful when you want to control access to an object, such as by adding security, logging, or lazy initialization, without modifying the original object's code. By using the proxy pattern, you can create a more flexible and maintainable codebase, as it allows you to add additional behavior to an object without modifying its class, adhering to the open-closed principle. This can be achieved by defining a proxy class that implements the same interface as the original object and holds a reference to an instance of the original object, controlling access to it and adding additional behavior as needed.

```
class Subject:
    def request(self):
        pass  # Interface for the original object

class RealSubject(Subject):
    def request(self):
        # Code for the original object's functionality
        pass

class Proxy(Subject):
    def __init__(self, real_subject):
        self.real_subject = real_subject  # Hold a reference to the original object

    def request(self):
        # Add additional behavior here (e.g., logging, security checks)
        return self.real_subject.request()  # Control access to the original object
```

```
public interface Subject {
```

(continues on next page)

(continued from previous page)

```
    void request(); // Interface for the original object
}

public class RealSubject implements Subject {
    @Override
    public void request() {
        // Code for the original object's functionality
    }
}

public class Proxy implements Subject {
    private RealSubject realSubject; // Hold a reference to the original object

    public Proxy(RealSubject realSubject) {
        this.realSubject = realSubject;
    }

    @Override
    public void request() {
        // Add additional behavior here (e.g., logging, security checks)
        realSubject.request(); // Control access to the original object
    }
}
```

Composite

Composite is a design pattern that composes objects into tree structures to represent part-whole hierarchies, allowing clients to treat individual objects and compositions of objects uniformly. This pattern is useful when you want to represent a hierarchy of objects, such as a file system or a graphical user interface, and you want to be able to treat individual objects and groups of objects in the same way. By using the composite pattern, you can create a more flexible and maintainable codebase, as it allows you to add new types of components without modifying existing code, adhering to the open-closed principle. This can be achieved by defining a component interface that declares methods for both individual objects and compositions, and then implementing that interface in both leaf classes (representing individual objects) and composite classes (representing groups of objects).

```
class Component:
    def operation(self):
        pass  # Interface for both individual objects and compositions

class Leaf(Component):
    def operation(self):
        # Code for the individual object's functionality
        pass

class Composite(Component):
    def __init__(self):
        self.children = []  # List to hold child components

    def add(self, component):
        self.children.append(component)  # Add a child component

    def remove(self, component):
        self.children.remove(component)  # Remove a child component

    def operation(self):
        for child in self.children:
```

(continues on next page)

(continued from previous page)

```
            child.operation()  # Forward the operation to child components
```

```
public interface Component {
    void operation(); // Interface for both individual objects and compositions
}

public class Leaf implements Component {
    @Override
    public void operation() {
        // Code for the individual object's functionality
    }
}

public class Composite implements Component {
    private List<Component> children = new ArrayList<>(); // List to hold child␣
↪components

    public void add(Component component) {
        children.add(component); // Add a child component
    }

    public void remove(Component component) {
        children.remove(component); // Remove a child component
    }

    @Override
    public void operation() {
        for (Component child : children) {
            child.operation(); // Forward the operation to child components
        }
    }
}
```

Behavioral patterns

Behavioral patterns are design patterns that focus on communication between objects, providing ways to define how objects interact and distribute responsibility. These patterns help to ensure that the interactions between objects are well-defined and organized, making it easier to understand and modify the behavior of a system. By using behavioral patterns in your software design, you can create a more flexible and maintainable codebase, as they provide a common language and structure for defining object interactions, making it easier to understand and modify the code in the future. Some common behavioral patterns include:

- **Observer:** Defines a one-to-many dependency between objects, so that when one object changes state, all its dependents are notified and updated automatically.
- **Strategy:** Defines a family of algorithms, encapsulates each one, and makes them interchangeable, allowing the algorithm to vary independently from clients that use it.
- **Command:** Encapsulates a request as an object, thereby allowing for parameterization of clients with queues, requests, and operations.
- **State:** Allows an object to alter its behavior when its internal state changes, appearing as if the object changed its class.
- **Template method:** Defines the skeleton of an algorithm in an operation, deferring some steps to subclasses, allowing them to redefine certain steps of the algorithm without changing its structure.
- **Iterator:** Provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

- **Mediator:** Defines an object that encapsulates how a set of objects interact, promoting loose coupling by keeping objects from referring to each other explicitly, and allowing their interaction to be varied independently.

Observer

Observer is a design pattern that defines a one-to-many dependency between objects, so that when one object changes state, all its dependents are notified and updated automatically. This pattern is useful when you want to create a system where multiple objects need to be informed of changes to a particular object, such as in a publish-subscribe model or an event-driven architecture. By using the observer pattern, you can create a more flexible and maintainable codebase, as it allows you to add new observers without modifying existing code, adhering to the open-closed principle. This can be achieved by defining an observer interface that declares a method for receiving updates, and then implementing that interface in concrete observer classes that register themselves with the subject they want to observe.

```python
class Subject:
    def __init__(self):
        self.observers = []  # List to hold registered observers

    def register_observer(self, observer):
        self.observers.append(observer)  # Register an observer

    def notify_observers(self):
        for observer in self.observers:
            observer.update()  # Notify all registered observers of a change

class Observer:
    def update(self):
        pass  # Method to receive updates from the subject

class ConcreteObserver(Observer):
    def update(self):
        # Code to handle the update from the subject
        pass
```

```java
public class Subject {
    private List<Observer> observers = new ArrayList<>(); // List to hold registered␣
↪observers

    public void registerObserver(Observer observer) {
        observers.add(observer); // Register an observer
    }

    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(); // Notify all registered observers of a change
        }
    }
}

public interface Observer {
    void update(); // Method to receive updates from the subject
}

public class ConcreteObserver implements Observer {
    @Override
    public void update() {
```

(continues on next page)

(continued from previous page)

```
        // Code to handle the update from the subject
    }
}
```

Strategy

Strategy is a design pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable, allowing the algorithm to vary independently from clients that use it. This pattern is useful when you want to create a system where multiple algorithms can be used interchangeably, such as in a sorting or compression context. By using the strategy pattern, you can create a more flexible and maintainable codebase, as it allows you to add new algorithms without modifying existing code, adhering to the open-closed principle. This can be achieved by defining a strategy interface that declares a method for executing the algorithm, and then implementing that interface in concrete strategy classes that provide specific implementations of the algorithm.

```python
class Strategy:
    def execute(self):
        pass  # Method to execute the algorithm

class ConcreteStrategyA(Strategy):
    def execute(self):
        # Code for algorithm A
        pass

class ConcreteStrategyB(Strategy):
    def execute(self):
        # Code for algorithm B
        pass
```

```java
public interface Strategy {
    void execute(); // Method to execute the algorithm
}

public class ConcreteStrategyA implements Strategy {
    @Override
    public void execute() {
        // Code for algorithm A
    }
}

public class ConcreteStrategyB implements Strategy {
    @Override
    public void execute() {
        // Code for algorithm B
    }
}
```

Command

Command is a design pattern that encapsulates a request as an object, thereby allowing for parameterization of clients with queues, requests, and operations. This pattern is useful when you want to create a system where requests can be treated as first-class objects, allowing for features such as undo/redo functionality, queuing of requests, or logging of operations. By using the command pattern, you can create a more flexible and maintainable codebase, as it allows you to add new commands without modifying existing code, adhering to the open-closed principle. This can be achieved by defining a command interface that declares a method for executing the command, and then implementing that interface in concrete command classes that encapsulate specific requests.

```python
class Command:
    def execute(self):
        pass  # Method to execute the command

class ConcreteCommandA(Command):
    def __init__(self, receiver):
        self.receiver = receiver  # Receiver of the command

    def execute(self):
        self.receiver.action_a()  # Execute the command by calling a method on the␣
↪receiver

class ConcreteCommandB(Command):
    def __init__(self, receiver):
        self.receiver = receiver  # Receiver of the command

    def execute(self):
        self.receiver.action_b()  # Execute the command by calling a method on the␣
↪receiver
```

```java
public interface Command {
    void execute(); // Method to execute the command
}

public class ConcreteCommandA implements Command {
    private Receiver receiver; // Receiver of the command

    public ConcreteCommandA(Receiver receiver) {
        this.receiver = receiver;
    }

    @Override
    public void execute() {
        receiver.actionA(); // Execute the command by calling a method on the receiver
    }
}

public class ConcreteCommandB implements Command {
    private Receiver receiver; // Receiver of the command

    public ConcreteCommandB(Receiver receiver) {
        this.receiver = receiver;
    }

    @Override
    public void execute() {
        receiver.actionB(); // Execute the command by calling a method on the receiver
    }
}
```

State

State is a design pattern that allows an object to alter its behavior when its internal state changes, appearing as if the object changed its class. This pattern is useful when you want to create a system where an object's behavior can change based on its state, such as in a finite state machine or a user interface context. By using the state pattern, you can create a more flexible and maintainable codebase, as it allows you to add new states without modifying existing code, adhering to the open-closed principle. This can be achieved by defining a state interface that declares

methods for handling requests, and then implementing that interface in concrete state classes that represent specific states of the object.

```python
class State:
    def handle(self):
        pass  # Method to handle requests based on the state

class ConcreteStateA(State):
    def handle(self):
        # Code for handling requests in state A
        pass

class ConcreteStateB(State):
    def handle(self):
        # Code for handling requests in state B
        pass
```

```java
public interface State {
    void handle(); // Method to handle requests based on the state
}

public class ConcreteStateA implements State {
    @Override
    public void handle() {
        // Code for handling requests in state A
    }
}

public class ConcreteStateB implements State {
    @Override
    public void handle() {
        // Code for handling requests in state B
    }
}
```

Template method

Template method is a design pattern that defines the skeleton of an algorithm in an operation, deferring some steps to subclasses, allowing them to redefine certain steps of the algorithm without changing its structure. This pattern is useful when you want to create a system where multiple algorithms share a common structure, but differ in specific steps, such as in a data processing pipeline or a game loop. By using the template method pattern, you can create a more flexible and maintainable codebase, as it allows you to add new algorithms without modifying existing code, adhering to the open-closed principle. This can be achieved by defining an abstract class that implements the template method, which calls abstract methods for the steps that can be overridden by subclasses.

```python
class AbstractClass:
    def template_method(self):
        self.step_one()  # Common step
        self.step_two()  # Common step
        self.step_three()  # Common step

    def step_one(self):
        pass  # Step one implementation

    def step_two(self):
        pass  # Step two implementation
```

(continues on next page)

(continued from previous page)

```python
    def step_three(self):
        pass  # Step three implementation

class ConcreteClassA(AbstractClass):
    def step_one(self):
        # Code for step one in algorithm A
        pass

    def step_two(self):
        # Code for step two in algorithm A
        pass

    def step_three(self):
        # Code for step three in algorithm A
        pass

class ConcreteClassB(AbstractClass):
    def step_one(self):
        # Code for step one in algorithm B
        pass

    def step_two(self):
        # Code for step two in algorithm B
        pass

    def step_three(self):
        # Code for step three in algorithm B
        pass
```

```java
public abstract class AbstractClass {
    public final void templateMethod() {
        stepOne(); // Common step
        stepTwo(); // Common step
        stepThree(); // Common step
    }

    protected abstract void stepOne(); // Step one implementation
    protected abstract void stepTwo(); // Step two implementation
    protected abstract void stepThree(); // Step three implementation
}

public class ConcreteClassA extends AbstractClass {
    @Override
    protected void stepOne() {
        // Code for step one in algorithm A
    }

    @Override
    protected void stepTwo() {
        // Code for step two in algorithm A
    }

    @Override
    protected void stepThree() {
        // Code for step three in algorithm A
    }
```

(continues on next page)

(continued from previous page)

```
}

public class ConcreteClassB extends AbstractClass {
    @Override
    protected void stepOne() {
        // Code for step one in algorithm B
    }

    @Override
    protected void stepTwo() {
        // Code for step two in algorithm B
    }

    @Override
    protected void stepThree() {
        // Code for step three in algorithm B
    }
}
```

Iterator

Iterator is a design pattern that provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation. This pattern is useful when you want to create a system where you need to traverse a collection of objects, such as in a list or a tree structure, and you want to be able to do so without exposing the internal structure of the collection. By using the iterator pattern, you can create a more flexible and maintainable codebase, as it allows you to add new types of collections without modifying existing code, adhering to the open-closed principle. This can be achieved by defining an iterator interface that declares methods for traversing the collection, and then implementing that interface in concrete iterator classes that provide specific implementations for different types of collections.

```
class Iterator:
    def has_next(self):
        pass  # Method to check if there are more elements to iterate over

    def next(self):
        pass  # Method to return the next element in the collection

class ConcreteIterator(Iterator):
    def __init__(self, collection):
        self.collection = collection  # Reference to the collection being iterated
        self.index = 0  # Current index in the iteration

    def has_next(self):
        return self.index < len(self.collection)  # Check if there are more elements

    def next(self):
        if self.has_next():
            element = self.collection[self.index]  # Get the next element
            self.index += 1  # Move to the next index
            return element
        else:
            raise StopIteration()  # No more elements to iterate over
```

```
public interface Iterator {
    boolean hasNext(); // Method to check if there are more elements to iterate over
    Object next(); // Method to return the next element in the collection
```

(continues on next page)

(continued from previous page)

```
}

public class ConcreteIterator implements Iterator {
    private List<Object> collection; // Reference to the collection being iterated
    private int index = 0; // Current index in the iteration

    public ConcreteIterator(List<Object> collection) {
        this.collection = collection;
    }

    @Override
    public boolean hasNext() {
        return index < collection.size(); // Check if there are more elements
    }

    @Override
    public Object next() {
        if (hasNext()) {
            Object element = collection.get(index); // Get the next element
            index++; // Move to the next index
            return element;
        } else {
            throw new NoSuchElementException(); // No more elements to iterate over
        }
    }
}
```

Mediator

Mediator is a design pattern that defines an object that encapsulates how a set of objects interact, promoting loose coupling by keeping objects from referring to each other explicitly, and allowing their interaction to be varied independently. This pattern is useful when you want to create a system where multiple objects need to communicate with each other, but you want to avoid tight coupling between them, such as in a chat application or a user interface context. By using the mediator pattern, you can create a more flexible and maintainable codebase, as it allows you to change the interactions between objects without modifying their classes, adhering to the open-closed principle. This can be achieved by defining a mediator interface that declares methods for communication between objects, and then implementing that interface in a concrete mediator class that coordinates the interactions between the objects.

```
class Mediator:
    def notify(self, sender, event):
        pass  # Method to handle communication between objects

class ConcreteMediator(Mediator):
    def __init__(self, component1, component2):
        self.component1 = component1  # Reference to component 1
        self.component2 = component2  # Reference to component 2
        self.component1.set_mediator(self)  # Set the mediator for component 1
        self.component2.set_mediator(self)  # Set the mediator for component 2

    def notify(self, sender, event):
        if sender == self.component1 and event == "EventA":
            # Handle event A from component 1 and coordinate with component 2
            pass
        elif sender == self.component2 and event == "EventB":
            # Handle event B from component 2 and coordinate with component 1
```

(continues on next page)

(continued from previous page)

```
            pass
```

```
public interface Mediator {
    void notify(Object sender, String event); // Method to handle communication␣
↪between objects
}

public class ConcreteMediator implements Mediator {
    private Component1 component1; // Reference to component 1
    private Component2 component2; // Reference to component 2

    public ConcreteMediator(Component1 component1, Component2 component2) {
        this.component1 = component1;
        this.component2 = component2;
        this.component1.setMediator(this); // Set the mediator for component 1
        this.component2.setMediator(this); // Set the mediator for component 2
    }

    @Override
    public void notify(Object sender, String event) {
        if (sender == component1 && event.equals("EventA")) {
            // Handle event A from component 1 and coordinate with component 2
        } else if (sender == component2 && event.equals("EventB")) {
            // Handle event B from component 2 and coordinate with component 1
        }
    }
}
```

2.1.13 OOP memory or architecture concepts

Memory management is an important aspect of object-oriented programming, as it involves the allocation and deallocation of memory for objects and data structures. In OOP, memory management can be handled manually by the programmer or automatically through garbage collection, depending on the programming language being used. Manual memory management requires the programmer to explicitly allocate and deallocate memory for objects, which can lead to issues such as memory leaks and dangling pointers if not done correctly. On the other hand, automatic memory management through garbage collection allows the programming language to automatically manage memory, freeing up resources when they are no longer needed, which can help to prevent memory-related bugs and improve the overall stability of the application. Understanding memory management concepts is crucial for writing efficient and reliable object-oriented code.

Garbage collection

Garbage collection is a form of automatic memory management that is used in many object-oriented programming languages, such as Java and Python. It works by automatically identifying and freeing up memory that is no longer being used by the program, which helps to prevent memory leaks and improve the overall stability of the application. Garbage collection typically involves a process of marking objects that are still in use and then sweeping through memory to reclaim the space occupied by objects that are no longer needed. This allows developers to focus on writing code without having to worry about manual memory management, while still ensuring that resources are efficiently utilized.

```
# Example of garbage collection in Python
class MyClass:
    def __init__(self, value):
        self.value = value

obj1 = MyClass(10)  # obj1 is created and occupies memory
```

(continues on next page)

(continued from previous page)

```
obj2 = MyClass(20)  # obj2 is created and occupies memory

del obj1  # obj1 is deleted, but memory is not immediately freed
# Garbage collector will eventually free the memory occupied by obj1
```

In Java, garbage collection is handled by the Java Virtual Machine (JVM), which automatically manages memory for Java applications. The JVM uses a generational garbage collection approach, where objects are categorized into different generations based on their age. The garbage collector periodically runs to identify and reclaim memory from objects that are no longer in use, allowing developers to focus on writing code without having to worry about manual memory management.

```
// Example of garbage collection in Java
public class MyClass {
    private int value;

    public MyClass(int value) {
        this.value = value;
    }
}

public class Main {
    public static void main(String[] args) {
        MyClass obj1 = new MyClass(10); // obj1 is created and occupies memory
        MyClass obj2 = new MyClass(20); // obj2 is created and occupies memory

        obj1 = null; // obj1 is dereferenced, but memory is not immediately freed
        // Garbage collector will eventually free the memory occupied by obj1
    }
}
```

Manual memory management

Manual memory management is a technique used in some programming languages, such as C and C++, where the programmer is responsible for explicitly allocating and deallocating memory for objects and data structures. This approach gives developers more control over memory usage, but it also introduces the risk of memory-related bugs, such as memory leaks (where allocated memory is not properly freed) and dangling pointers (where a pointer references memory that has already been deallocated). To avoid these issues, programmers must carefully manage memory by ensuring that every allocation has a corresponding deallocation, and by using tools such as smart pointers or memory management libraries to help automate this process. Manual memory management can be more efficient in terms of performance, but it requires a higher level of attention to detail and can lead to more complex code compared to automatic memory management through garbage collection.

```
// Example of manual memory management in C
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int value;
} MyStruct;

int main() {
    MyStruct* obj1 = (MyStruct*)malloc(sizeof(MyStruct)); // Allocate memory for obj1
    obj1->value = 10; // Use obj1

    MyStruct* obj2 = (MyStruct*)malloc(sizeof(MyStruct)); // Allocate memory for obj2
    obj2->value = 20; // Use obj2
```

(continues on next page)

(continued from previous page)

```
    free(obj1); // Deallocate memory for obj1
    free(obj2); // Deallocate memory for obj2

    return 0;
}
```

Attention

Manual memory management can lead to issues such as memory leaks and dangling pointers if not done correctly, so it is important for developers to be diligent in managing memory when using this approach.

In Python and Java, memory management is handled automatically through garbage collection, so developers do not need to worry about manual memory management. However, it is still important to be mindful of memory usage and to avoid creating unnecessary objects or holding onto references that are no longer needed, as this can lead to increased memory usage and potential performance issues.

In C and C++, developers must be careful to ensure that every allocation of memory has a corresponding deallocation, and to avoid common pitfalls such as double freeing (where memory is deallocated more than once) and use-after-free (where memory is accessed after it has been deallocated). Using tools such as smart pointers in C++ can help to automate memory management and reduce the risk of these issues.

```
// Example of using smart pointers in C++
#include <iostream>
#include <memory>

class MyClass {
public:
    MyClass(int value) : value(value) {}
    void display() { std::cout << "Value: " << value << std::endl; }
private:
    int value;
};

int main() {
    std::unique_ptr<MyClass> obj1 = std::make_unique<MyClass>(10); // Automatically␣
↪managed memory
    obj1->display(); // Use obj1

    std::unique_ptr<MyClass> obj2 = std::make_unique<MyClass>(20); // Automatically␣
↪managed memory
    obj2->display(); // Use obj2

    // No need to manually free memory, it will be automatically deallocated when the␣
↪unique_ptr goes out of scope
    return 0;
}
```

Object identity

Object identity is a fundamental concept in object-oriented programming that refers to the unique identity of an object, which distinguishes it from other objects. In OOP, each object has its own identity, which is typically represented by a memory address or a unique identifier. This allows objects to be compared for equality based on their identity rather than their state or value. Object identity is important for maintaining the integrity of data and ensuring that operations on objects are performed correctly, as it allows developers to determine whether two references point to the same object in memory. Understanding object identity is crucial for writing effective and

efficient object-oriented code, as it can impact how objects are managed and manipulated within a program.

```
class MyClass:
    def __init__(self, value):
        self.value = value

obj1 = MyClass(10)  # obj1 is created and has a unique identity
obj2 = MyClass(10)  # obj2 is created and has a different unique identity

print(obj1 == obj2)  # Output: False (obj1 and obj2 are different objects)
print(obj1 is obj2)  # Output: False (obj1 and obj2 do not have the same identity)

obj3 = obj1  # obj3 references the same object as obj1
print(obj1 == obj3)  # Output: True (obj1 and obj3 are the same object)
print(obj1 is obj3)  # Output: True (obj1 and obj3 have the same identity)
```

```
public class MyClass {
    private int value;

    public MyClass(int value) {
        this.value = value;
    }
}

public class Main {
    public static void main(String[] args) {
        MyClass obj1 = new MyClass(10); // obj1 is created and has a unique identity
        MyClass obj2 = new MyClass(10); // obj2 is created and has a different unique␣
↪identity

        System.out.println(obj1.equals(obj2)); // Output: false (obj1 and obj2 are␣
↪different objects)
        System.out.println(obj1 == obj2); // Output: false (obj1 and obj2 do not have␣
↪the same identity)

        MyClass obj3 = obj1; // obj3 references the same object as obj1
        System.out.println(obj1.equals(obj3)); // Output: true (obj1 and obj3 are the␣
↪same object)
        System.out.println(obj1 == obj3); // Output: true (obj1 and obj3 have the␣
↪same identity)
    }
}
```

Shallow copy vs. deep copy

Copying objects in object-oriented programming can be done in two ways: shallow copy and deep copy. A shallow copy creates a new object that is a copy of the original object, but it only copies the references to the objects contained within the original object. This means that if the original object contains mutable objects (such as lists or dictionaries), both the original and the shallow copy will reference the same mutable objects, and changes to those objects will affect both the original and the copy. On the other hand, a deep copy creates a new object that is a copy of the original object, along with copies of all the objects contained within it. This means that changes to mutable objects in the deep copy will not affect the original object, as they are completely independent. Understanding the difference between shallow and deep copying is important for managing memory and ensuring that your code behaves as expected when working with complex data structures.

```
import copy
```

(continues on next page)

(continued from previous page)

```python
class MyClass:
    def __init__(self, value):
        self.value = value

original = MyClass([1, 2, 3])  # Original object with a mutable list
shallow_copy = copy.copy(original)  # Shallow copy of the original object
deep_copy = copy.deepcopy(original)  # Deep copy of the original object

shallow_copy.value.append(4)  # Modifying the mutable list in the shallow copy
print(original.value)  # Output: [1, 2, 3, 4] (original is affected by changes to␣
↪shallow copy)
print(deep_copy.value)  # Output: [1, 2, 3] (deep copy is unaffected by changes to␣
↪shallow copy)

deep_copy.value.append(5)  # Modifying the mutable list in the deep copy
print(original.value)  # Output: [1, 2, 3, 4] (original is unaffected by changes to␣
↪deep copy)
print(deep_copy.value)  # Output: [1, 2, 3, 5] (deep copy is modified independently␣
↪of original)
```

Attention

In Java, the concept of shallow and deep copying can be implemented using the `clone()` method for shallow copying and by implementing a custom method for deep copying, as Java does not provide built-in support for deep copying. It is important to be mindful of how objects are copied in Java, especially when dealing with mutable objects, to avoid unintended side effects.

```java
public class MyClass implements Cloneable {
    private List<Integer> value;

    public MyClass(List<Integer> value) {
        this.value = value;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone(); // Shallow copy
    }

    public MyClass deepCopy() {
        List<Integer> newValue = new ArrayList<>(this.value); // Deep copy of the list
        return new MyClass(newValue);
    }
}
```

Immutability

Immutabiltiy is a design principle in object-oriented programming that refers to the state of an object being unchangeable after it has been created. An immutable object is one whose state cannot be modified after it has been instantiated, meaning that any changes to the object's state result in the creation of a new object rather than modifying the existing one. Immutability can help to improve the safety and predictability of code, as it eliminates issues related to shared mutable state and makes it easier to reason about the behavior of objects. In languages like Java, immutability can be achieved by declaring fields as final and not providing setter methods, while in Python, immutability can be achieved by using tuples or by defining classes with read-only properties.

```python
class ImmutableClass:
    def __init__(self, value):
        self._value = value  # Private attribute to store the value

    @property
    def value(self):
        return self._value  # Read-only property to access the value

obj1 = ImmutableClass(10)  # Create an immutable object
print(obj1.value)  # Output: 10

# Attempting to modify the value will result in an error
# obj1.value = 20  # This will raise an AttributeError
```

```java
public final class ImmutableClass {
    private final int value; // Final field to store the value

    public ImmutableClass(int value) {
        this.value = value; // Initialize the value in the constructor
    }

    public int getValue() {
        return value; // Getter method to access the value
    }
}

public class Main {
    public static void main(String[] args) {
        ImmutableClass obj1 = new ImmutableClass(10); // Create an immutable object
        System.out.println(obj1.getValue()); // Output: 10

        // Attempting to modify the value will result in a compilation error
        // obj1.value = 20; // This will cause a compilation error due to the final␣
↪field
    }
}
```

2.1.14 Error handling and contracts

Error handling is an important aspect of object-oriented programming, as it allows developers to manage and respond to exceptional conditions that may arise during the execution of a program. In OOP, error handling can be implemented using techniques such as try-catch blocks, custom exceptions, and assertions. By using these techniques, developers can create more robust and reliable code that can gracefully handle errors and prevent crashes or unexpected behavior. Additionally, contracts can be used to define the expected behavior of methods and classes, ensuring that they are used correctly and that any violations of the contract are properly handled. Understanding error handling and contracts is crucial for writing effective and maintainable object-oriented code.

Exception handling

Exception handling is a mechanism in object-oriented programming that allows developers to manage and respond to errors or exceptional conditions that may occur during the execution of a program. This is typically done using try-catch blocks, where code that may throw an exception is placed within a try block, and the corresponding catch block is used to handle the exception if it occurs. Exception handling helps to improve the robustness and reliability of code by allowing developers to gracefully handle errors and prevent crashes or unexpected behavior. Additionally, custom exceptions can be created to provide more specific error information and to allow for more fine-grained error handling.

Exceptions are typically used to signal that an error has occurred, and they can be caught and handled by the

program to prevent it from crashing. In Java, exceptions are categorized into checked exceptions (which must be declared in the method signature) and unchecked exceptions (which do not need to be declared). In Python, all exceptions are unchecked, and developers can create custom exception classes by inheriting from the built-in Exception class.

Several exception includes:

- **FileNotFoundError:** Raised when a file or directory is requested but cannot be found.
- **ValueError:** Raised when a built-in operation or function receives an argument that has the right type but an inappropriate value.
- **TypeError:** Raised when an operation or function is applied to an object of inappropriate type.
- **IndexError:** Raised when a sequence subscript is out of range.
- **KeyError:** Raised when a mapping (dictionary) key is not found in the set of existing keys.
- **ZeroDivisionError:** Raised when the second argument of a division or modulo operation is zero.
- **Custom exceptions:** Developers can create their own exception classes to represent specific error conditions in their applications, allowing for more precise error handling and improved code readability.

There are several best practices for exception handling in object-oriented programming, including:

- Catching specific exceptions rather than using a generic catch-all approach, to ensure that errors are handled appropriately and to avoid masking other issues.
- Providing meaningful error messages and logging information to help with debugging and troubleshooting.
- Avoiding the use of exceptions for control flow, as this can lead to code that is difficult to read and maintain.
- Ensuring that resources are properly released in the event of an exception, such as by using finally blocks or context managers to manage resource cleanup.

Attention

It is important to use exception handling judiciously, as overusing exceptions or using them inappropriately can lead to code that is difficult to read and maintain. Developers should strive to write code that is robust and handles errors gracefully, while also ensuring that exceptions are used in a way that enhances the clarity and maintainability of the codebase.

```java
public class Main {
    public static void main(String[] args) {
        try {
            // Code that may throw an exception
            int result = 10 / 0; // This will throw a ZeroDivisionError
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage()); // Handle the exception
        } finally {
            System.out.println("This block will always execute."); // Resource␣
↪cleanup or final actions
        }
    }
}
```

```python
try:
    # Code that may throw an exception
    result = 10 / 0  # This will raise a ZeroDivisionError
except ZeroDivisionError as e:
    print(f"Error: {e}")  # Handle the exception
```

(continues on next page)

(continued from previous page)

```
finally:
    print("This block will always execute.")  # Resource cleanup or final actions
```

Custom exceptions

Some exceptions are built into programming languages, but developers can also create their own custom exceptions to represent specific error conditions in their applications. Custom exceptions can be created by defining a new class that inherits from the built-in Exception class (or a relevant subclass) and adding any additional attributes or methods as needed. This allows developers to provide more specific error information and to allow for more fine-grained error handling in their code. For example, in Python, you can create a custom exception like this:

```
class CustomError(Exception):
    def __init__(self, message):
        super().__init__(message)  # Call the base class constructor with the error␣
↪message

# Example usage of the custom exception
try:
    raise CustomError("This is a custom error message.")  # Raise the custom exception
except CustomError as e:
    print(f"Caught a custom exception: {e}")  # Handle the custom exception
```

In Java, you can create a custom exception by defining a new class that extends the Exception class (or a relevant subclass) and providing a constructor to initialize the error message. For example:

```
public class CustomException extends Exception {
    public CustomException(String message) {
        super(message); // Call the base class constructor with the error message
    }
}

// Example usage of the custom exception
public class Main {
    public static void main(String[] args) {
        try {
            throw new CustomException("This is a custom error message."); // Raise␣
↪the custom exception
        } catch (CustomException e) {
            System.out.println("Caught a custom exception: " + e.getMessage()); //␣
↪Handle the custom exception
        }
    }
}
```

Custom exceptions can be particularly useful for representing specific error conditions that are relevant to the domain of the application, allowing for more precise error handling and improved code readability. By using custom exceptions, developers can create a more robust and maintainable codebase that can effectively manage and respond to errors in a way that is meaningful to the context of the application.

Contracts and assertions

Contracts and assertions are techniques used in object-oriented programming to define the expected behavior of methods and classes, ensuring that they are used correctly and that any violations of the contract are properly handled. A contract is a formal agreement between a method or class and its callers, specifying the preconditions (what must be true before the method is called), postconditions (what must be true after the method is called), and invariants (conditions that must always hold true for the class). Assertions are statements that check for specific conditions at runtime, and if an assertion fails, it typically raises an error or exception. By using contracts and

assertions, developers can improve the reliability and maintainability of their code by catching errors early and ensuring that methods and classes are used in a way that adheres to their intended design. For example, in Python, you can use assertions to check for preconditions and postconditions like this:

```
def divide(a, b):
    assert b != 0, "Precondition failed: denominator must not be zero"  # Precondition
    result = a / b
    assert result >= 0, "Postcondition failed: result must be non-negative"  #␣
↪Postcondition
    return result

# Example usage of the divide function
try:
    print(divide(10, 2))  # This will work and print 5.0
    print(divide(10, 0))  # This will raise an assertion error due to the precondition
except AssertionError as e:
    print(f"Assertion error: {e}")
```

In Java, you can use assertions to check for conditions at runtime by using the assert keyword. For example:

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;

        assert b != 0 : "Precondition failed: denominator must not be zero"; //␣
↪Precondition
        int result = a / b; // This will throw an ArithmeticException if b is zero
        assert result >= 0 : "Postcondition failed: result must be non-negative"; //␣
↪Postcondition

        System.out.println(result);
    }
}
```

For contracts, you can use design by contract principles to define the expected behavior of methods and classes, and you can use tools such as JML (Java Modeling Language) for Java or PyContracts for Python to specify contracts in a more formal way. By using contracts and assertions effectively, developers can create more robust and maintainable code that adheres to the intended design and can catch errors early in the development process.

```
// Example of using JML for design by contract in Java
public class Calculator {
    //@ requires b != 0; // Precondition: denominator must not be zero
    //@ ensures \result >= 0; // Postcondition: result must be non-negative
    public int divide(int a, int b) {
        return a / b; // This will throw an ArithmeticException if b is zero
    }
}
```

```
# Example of using PyContracts for design by contract in Python
from contracts import contract

@contract
def divide(a: int, b: int) -> int:
    """Divide two integers with preconditions and postconditions."""
    assert b != 0, "Precondition failed: denominator must not be zero"  # Precondition
    result = a // b
    assert result >= 0, "Postcondition failed: result must be non-negative"  #␣
```

(continues on next page)

(continued from previous page)

```
↪Postcondition
    return result
```

2.1.15 Modern OOP extensions

In modern object-oriented programming, there are several extensions and features that have been introduced to enhance the capabilities of OOP languages and to provide developers with more tools for writing efficient and maintainable code. Some of these modern OOP extensions include:

- **Annotations:** Annotations are a way to provide metadata about code elements such as classes, methods, and fields. They can be used for various purposes, such as documentation, code analysis, and runtime processing. Annotations allow developers to add additional information to their code without affecting its functionality, making it easier to understand and maintain.
- **Reflection:** Reflection is a feature that allows a program to inspect and modify its own structure and behavior at runtime. This can be useful for tasks such as dynamically loading classes, invoking methods, and accessing fields. Reflection provides a powerful way to create flexible and adaptable code, but it should be used with caution as it can lead to performance issues and security vulnerabilities if not used properly.
- **Generics:** Generics are a feature that allows developers to create classes, interfaces, and methods that can operate on different types of data while providing type safety. Generics enable code reuse and can help to reduce the need for type casting, making code more readable and maintainable. They are commonly used in collections and data structures to allow for type-safe operations on different types of objects.
- **Dependency injection:** Dependency injection is a design pattern that allows for the decoupling of components in a software system by injecting dependencies into a class rather than having the class create its own dependencies. This promotes loose coupling and makes it easier to test and maintain code, as dependencies can be easily swapped out or mocked during testing.
- **Inversion of control:** Inversion of control (IoC) is a design principle that refers to the reversal of the flow of control in a software system. Instead of a class controlling its own dependencies and behavior, control is inverted and delegated to an external entity, such as a framework or container. This allows for greater flexibility and modularity in code, as it promotes separation of concerns and makes it easier to manage dependencies and interactions between components.

Annotations

Annotations are a way to provide metadata about code elements such as classes, methods, and fields. They can be used for various purposes, such as documentation, code analysis, and runtime processing. Annotations allow developers to add additional information to their code without affecting its functionality, making it easier to understand and maintain. In Java, annotations are defined using the @interface keyword and can be applied to various code elements using the @ symbol. In Python, annotations can be added using decorators or by using type hints to provide information about the expected types of function parameters and return values.

```
// Example of using annotations in Java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnnotation {
    String value(); // Define an annotation with a value element
}

public class MyClass {
    @MyAnnotation("This is a custom annotation") // Apply the annotation to a method
    public void myMethod() {
        // Method implementation
    }
}
```

```python
# Example of using annotations in Python
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("This is a custom annotation")  # Add metadata or behavior to the␣
↪function
        return func(*args, **kwargs)
    return wrapper

@my_decorator  # Apply the decorator as an annotation to the function
def my_function():
    print("This is the original function.")

my_function()  # This will print the annotation message followed by the original␣
↪function message
```

Reflection

Reflection is a feature that allows a program to inspect and modify its own structure and behavior at runtime. This can be useful for tasks such as dynamically loading classes, invoking methods, and accessing fields. Reflection provides a powerful way to create flexible and adaptable code, but it should be used with caution as it can lead to performance issues and security vulnerabilities if not used properly. In Java, reflection is provided through the java.lang.reflect package, while in Python, reflection can be achieved using built-in functions such as getattr(), setattr(), and the inspect module.

```java
// Example of using reflection in Java
import java.lang.reflect.Method;

public class Main {
    public static void main(String[] args) {
        try {
            Class<?> clazz = Class.forName("MyClass"); // Dynamically load the MyClass
            Method method = clazz.getMethod("myMethod"); // Get the myMethod from␣
↪MyClass
            method.invoke(clazz.newInstance()); // Invoke the myMethod on an instance␣
↪of MyClass
        } catch (Exception e) {
            e.printStackTrace(); // Handle any exceptions that may occur
        }
    }
}
```

```python
// Example of using reflection in Python
class MyClass:
    def my_method(self):
        print("This is my_method in MyClass.")

# Dynamically load the MyClass and invoke my_method
class_name = "MyClass"
method_name = "my_method"
clazz = globals()[class_name]  # Get the class from the global namespace
method = getattr(clazz(), method_name)  # Get the method from the class instance
method()  # Invoke the method, which will print the message from my_method
```

Generics

Generics are a feature that allows developers to create classes, interfaces, and methods that can operate on different types of data while providing type safety. Generics enable code reuse and can help to reduce the need for type casting, making code more readable and maintainable. They are commonly used in collections and data structures to allow for type-safe operations on different types of objects. In Java, generics are implemented using angle brackets (<>), while in Python, generics can be achieved using type hints and the typing module.

```java
// Example of using generics in Java
public class Box<T> {
    private T content; // Generic type parameter

    public void setContent(T content) {
        this.content = content; // Set the content of the box
    }

    public T getContent() {
        return content; // Get the content of the box
    }
}

public class Main {
    public static void main(String[] args) {
        Box<String> stringBox = new Box<>(); // Create a box for strings
        stringBox.setContent("Hello, Generics!"); // Set the content of the string box
        System.out.println(stringBox.getContent()); // Output: Hello, Generics!

        Box<Integer> integerBox = new Box<>(); // Create a box for integers
        integerBox.setContent(42); // Set the content of the integer box
        System.out.println(integerBox.getContent()); // Output: 42
    }
}
```

```python
# Example of using generics in Python with type hints
from typing import TypeVar, Generic

T = TypeVar('T')  # Define a generic type variable

class Box(Generic[T]):
    def __init__(self):
        self.content: T = None  # Initialize content with the generic type

    def set_content(self, content: T) -> None:
        self.content = content  # Set the content of the box

    def get_content(self) -> T:
        return self.content  # Get the content of the box

string_box = Box[str]()  # Create a box for strings
string_box.set_content("Hello, Generics!")  # Set the content of the string box
print(string_box.get_content())  # Output: Hello, Generics!
```

Dependency injection

Dependencies are objects or components that a class relies on to perform its functionality. In object-oriented programming, managing dependencies is crucial for creating modular and maintainable code. One common approach to managing dependencies is through the use of dependency injection, which allows for the decoupling of components in a software system by injecting dependencies into a class rather than having the class create its own

dependencies. This promotes loose coupling and makes it easier to test and maintain code, as dependencies can be easily swapped out or mocked during testing. In Java, dependency injection can be implemented using frameworks such as Spring or Guice, while in Python, it can be achieved using libraries such as dependency-injector or by manually passing dependencies through constructors or function parameters.

Dependency injection is a design pattern that allows for the decoupling of components in a software system by injecting dependencies into a class rather than having the class create its own dependencies. This promotes loose coupling and makes it easier to test and maintain code, as dependencies can be easily swapped out or mocked during testing. In Java, dependency injection can be implemented using frameworks such as Spring or Guice, while in Python, it can be achieved using libraries such as dependency-injector or by manually passing dependencies through constructors or function parameters.

Dependency injection can be implemented in various ways, including constructor injection (where dependencies are provided through the class constructor), setter injection (where dependencies are provided through setter methods), and interface injection (where dependencies are provided through an interface). By using dependency injection, developers can create more modular and flexible code that is easier to test and maintain, as it allows for the separation of concerns and promotes the use of interfaces and abstractions. For example, in Java, you can use constructor injection with the Spring framework like this:

```java
// Example of using dependency injection with Spring in Java
@Component
public class MyService {
    private final MyRepository repository; // Dependency to be injected

    @Autowired
    public MyService(MyRepository repository) {
        this.repository = repository; // Constructor injection of the dependency
    }

    public void performAction() {
        repository.doSomething(); // Use the injected dependency
    }
}

@Component
public class MyRepository {
    public void doSomething() {
        // Implementation of the repository method
    }
}
```

```python
# Example of using dependency injection in Python
class MyService:
    def __init__(self, repository):
        self.repository = repository  # Constructor injection of the dependency

    def perform_action(self):
        self.repository.do_something()  # Use the injected dependency

class MyRepository:
    def do_something(self):
        # Implementation of the repository method
        pass

# Manually injecting the dependency
repository = MyRepository()  # Create an instance of the repository
service = MyService(repository)  # Inject the repository into the service
service.perform_action()  # Call a method on the service that uses the injected␣
↪dependency
```

Inversion of control

Control is a fundamental principle in software design that refers to the flow of execution in a program. In traditional programming, the flow of control is typically determined by the program itself, with the main function or entry point controlling the execution of the program. However, in object-oriented programming, there is a design principle called inversion of control (IoC) that refers to the reversal of the flow of control in a software system. Instead of a class controlling its own dependencies and behavior, control is inverted and delegated to an external entity, such as a framework or container. This allows for greater flexibility and modularity in code, as it promotes separation of concerns and makes it easier to manage dependencies and interactions between components. Inversion of control can be implemented using various techniques, such as dependency injection, event-driven programming, and service locators. By using inversion of control, developers can create more flexible and adaptable code that can easily accommodate changes in requirements and promote better separation of concerns. For example, in Java, you can use the Spring framework to implement inversion of control like this:

```java
// Example of using inversion of control with Spring in Java
@Component
public class MyService {
    private final MyRepository repository; // Dependency to be injected

    @Autowired
    public MyService(MyRepository repository) {
        this.repository = repository; // Constructor injection of the dependency
    }

    public void performAction() {
        repository.doSomething(); // Use the injected dependency
    }
}

@Component
public class MyRepository {
    public void doSomething() {
        // Implementation of the repository method
    }
}
```

```python
# Example of using inversion of control in Python
class MyService:
    def __init__(self, repository):
        self.repository = repository  # Constructor injection of the dependency

    def perform_action(self):
        self.repository.do_something()  # Use the injected dependency

class MyRepository:
    def do_something(self):
        # Implementation of the repository method
        pass

# Manually injecting the dependency
repository = MyRepository()  # Create an instance of the repository
service = MyService(repository)  # Inject the repository into the service
service.perform_action()  # Call a method on the service that uses the injected␣
↪dependency
```

2.2 Appendix B: MVC Architecture

2.2.1 Introduction

Conceptual foundation of MVC

Model-View-Controller (MVC) is an architectural pattern developed to address a fundamental structural problem in software engineering: the uncontrolled mixing of data management, presentation logic, and control flow. At its most basic level, MVC divides an application into three interacting components — the Model, the View, and the Controller — but the importance of this division lies not in taxonomy, but in responsibility control.

In many early-stage implementations, developers tend to write code in a way that mirrors immediate functionality. A button click might validate data, update system state, call a hardware function, and update the display — all within the same method. While this appears efficient in small systems, such entanglement scales poorly. The resulting architecture exhibits high coupling and low cohesion, making modification increasingly risky as the system grows.

MVC addresses this by enforcing a deliberate separation:

- The **Model** owns system state and business logic
- The **View** is responsible for presenting information
- The **Controller** mediates interaction between the two

This structural separation became widely adopted in large-scale frameworks such as Django and ASP.NET MVC precisely because long-term maintainability demands architectural discipline. MVC is therefore not merely a coding style; it is a strategy for managing complexity over time.

Why Separation is architecturally necessary

To understand the necessity of MVC, consider a simplified Android example in which a movement button directly manipulates robot state:

```
moveButton.setOnClickListener(v -> {
    if (batteryLevel > 10) {
        robot.moveForward();
        batteryLevel -= 5;
        batteryText.setText("Battery: " + batteryLevel);
    }
});
```

At first glance, this implementation appears straightforward. However, several structural problems emerge:

- Battery validation logic is embedded in the UI layer
- Domain state is directly modified by presentation code
- Hardware commands are triggered from within a UI callback
- Display updates are interwoven with business logic

This design increases coupling across unrelated concerns. If battery rules change, the UI must be modified. If hardware behavior changes, UI code must be updated. If the interface is redesigned, domain logic risks accidental modification. The architecture lacks clear boundaries.

MVC restructures this situation by isolating responsibilities. Instead of allowing the View to manipulate state directly, it forwards input to a Controller, which delegates domain logic to the Model. This indirection may appear unnecessary in small examples, but in larger systems — especially distributed or safety-sensitive systems — it significantly reduces systemic fragility.

2.2.2 Core concepts

The Model: authority, integrity, and cohesion

The Model is not simply a data container; it is the authoritative representation of domain knowledge. Its purpose is to maintain state consistently and enforce invariants. In a robotic system, invariants might include battery limits, safety constraints, or operational states such as emergency stop.

Consider the following example:

```
public class RobotState {
    private int batteryLevel = 100;

    public boolean canMove() {
        return batteryLevel > 10;
    }

    public void consumeBattery(int amount) {
        if (batteryLevel - amount < 0) {
            throw new IllegalStateException("Battery depleted");
        }

        batteryLevel -= amount;
    }

    public int getBatteryLevel() {
        return batteryLevel;
    }
}
```

In this implementation, all battery-related logic resides within the Model. This centralization has several architectural implications.

First, it increases cohesion. All behavior related to battery management is encapsulated in one module. Second, it reduces duplication; validation logic is written once rather than scattered across multiple UI handlers. Third, it enhances testability. The Model can be tested independently of Android or ROS2 frameworks.

Invariant centralization improves correctness guarantees. When validation is distributed, inconsistencies are likely. When centralized, the Model becomes the single source of truth. In robotics, where state integrity affects safety, this centralization is not merely beneficial — it is essential.

The View: representation without decision authority

The View is responsible for rendering information and capturing user input, but it intentionally lacks decision-making authority regarding domain rules. This restriction may appear limiting, yet it is critical for architectural stability.

For example:

```
batteryText.setText("Battery: " + robotState.getBatteryLevel());

moveButton.setOnClickListener(v -> {
    controller.onMoveRequested();
});
```

Here, the View displays state retrieved from the Model and forwards input to the Controller. It does not decide whether the robot can move; it merely delegates that responsibility.

The argument for limiting View authority lies in change volatility. User interfaces change frequently due to usability improvements, layout redesigns, or platform evolution. If domain logic is embedded in the View, every UI modification risks altering system behavior. By isolating presentation from logic, MVC reduces change propagation and increases maintainability.

This separation aligns with principles of low coupling and high cohesion, ensuring that aesthetic or usability changes do not destabilize system rules.

The Controller: coordination without ownership

The Controller functions as a mediator. It receives user input from the View, invokes operations on the Model, and updates the View accordingly.

```
public void onMoveRequested() {
    if (robotState.canMove()) {
        robotState.consumeBattery(5);
        robot.moveForward();
        View.updateBattery(robotState.getBatteryLevel());
    }
}
```

The Controller does not enforce battery rules independently; it delegates that responsibility to the Model. Its role is coordination, not ownership.

However, Controllers are often the most vulnerable component in MVC. Developers may gradually accumulate logic within them, leading to "fat Controllers." When Controllers begin implementing validation, managing persistent state, or performing domain computations, the separation collapses.

Thus, MVC requires disciplining. The Controller must remain a mediator, not an alternative domain layer.

Interaction flow and dependency direction

MVC establishes a clear flow of interaction:

- The View captures user input
- The Controller interprets the request
- The Model performs state updates
- The Controller updates the View

Critically, the Model does not depend on the View. This unidirectional dependency structure prevents circular references and preserves independence from presentation frameworks.

In distributed Android-ROS2 systems, this separation has practical implications. The Model can represent robot state independent of Android lifecycle events. Network disruptions affect coordination logic but not domain integrity. UI crashes do not compromise robotic safety rules.

Architecturally, this improves fault isolation and resilience.

2.2.3 Analysis

While MVC improves structural clarity, it is not without cost.

First, it introduces additional abstraction layers. Small projects may perceive this as unnecessary overhead. The architectural benefit emerges primarily in systems of moderate or large complexity. Thus, MVC represents a trade-off between immediate simplicity and long-term maintainability.

Second, the reliance on Controllers as coordination hubs introduces a risk of logic centralization. Without careful boundary enforcement, Controllers may grow excessively complex. This risk must be mitigated through disciplined design practices.

Third, unlike reactive frameworks such as React, traditional MVC requires explicit updates to the View. This increases verbosity but provides transparency. The trade-off is between automated reactivity and explicit control.

Finally, additional abstraction layers may introduce minor performance overhead. In robotics systems where latency matters, this must be considered. However, coupling UI directly to hardware for micro-optimizations would undermine safety and maintainability. In safety-critical environments, architectural integrity outweighs marginal performance gains.

2.3 Appendix C: MVVM Architecture

2.3.1 Introduction

Conceptual Foundation of MVVM

Model–View–ViewModel (MVVM) emerged as an evolution of Model–View–Controller, motivated by a practical limitation observed in GUI-based systems: as user interfaces became more dynamic and state-driven, Controllers increasingly accumulated responsibilities related to synchronization between state and presentation.

MVVM restructures this coordination responsibility by introducing an intermediate abstraction — the **ViewModel** — whose purpose is to expose application state in a form directly consumable by the View.

The fundamental claim of MVVM is that synchronization between UI and state should be declarative rather than imperative. Instead of manually instructing the View when to update, the View binds to observable state, and updates propagate automatically when that state changes.

This paradigm gained prominence in UI-centric frameworks such as Angular and is formally supported in Android's Jetpack architecture components.

MVVM therefore reflects a broader architectural shift toward reactive programming and state-driven UI design.

The Problem MVVM Attempts to Solve

In traditional MVC, Controllers are responsible for updating the View whenever the Model changes:

```
public void onMoveRequested() {
    if (robotState.canMove()) {
        robotState.consumeBattery(5);
        view.updateBattery(robotState.getBatteryLevel());
    }
}
```

This explicit synchronization introduces several architectural concerns:

1. The Controller must know which UI elements depend on which state.
2. UI updates are manually triggered, increasing cognitive load.
3. The system risks inconsistency if update calls are forgotten.
4. The Controller becomes responsible for presentation formatting.

As applications scale and UI complexity increases, these responsibilities accumulate. The Controller gradually becomes both coordination and synchronization manager, which weakens separation of concerns.

MVVM addresses this by moving synchronization logic into a ViewModel layer that exposes observable state representations.

2.3.2 Component Structure of MVVM

The Model

The Model in MVVM retains the same conceptual role as in MVC: it encapsulates domain logic, invariants, and authoritative state.

For example:

```
public class RobotState {

    private int batteryLevel = 100;

    public boolean canMove() {
        return batteryLevel > 10;
```

(continues on next page)

(continued from previous page)

```
    }

    public void consumeBattery(int amount) {
        if (batteryLevel - amount < 0) {
            throw new IllegalStateException("Battery depleted");
        }
        batteryLevel -= amount;
    }

    public int getBatteryLevel() {
        return batteryLevel;
    }
}
```

The Model remains independent of Android lifecycle classes and UI frameworks. This independence preserves testability and portability.

Architecturally, MVVM does not weaken domain isolation; it introduces an additional transformation layer above it.

The ViewModel

The ViewModel is the defining element of MVVM. It does not render UI and does not directly manipulate Views. Instead, it:

- Exposes observable state derived from the Model.
- Transforms domain data into UI-ready representations.
- Handles user intents.
- Survives configuration changes (in Android).

Example using Android LiveData:

```
public class RobotViewModel extends ViewModel {

    private RobotState robotState = new RobotState();
    private MutableLiveData<Integer> batteryLiveData = new MutableLiveData<>();

    public RobotViewModel() {
        batteryLiveData.setValue(robotState.getBatteryLevel());
    }

    public LiveData<Integer> getBatteryLevel() {
        return batteryLiveData;
    }

    public void onMoveRequested() {
        if (robotState.canMove()) {
            robotState.consumeBattery(5);
            batteryLiveData.setValue(robotState.getBatteryLevel());
        }
    }
}
```

The ViewModel exposes state in observable form. When *batteryLiveData* changes, the View automatically reacts.

This design removes manual synchronization responsibilities from Controllers and shifts them into a structured reactive layer.

Architecturally, the ViewModel becomes a state adapter between domain logic and UI.

The View

The View binds to observable data:

```
viewModel.getBatteryLevel().observe(this, level -> {
    batteryText.setText("Battery: " + level);
});

moveButton.setOnClickListener(v -> {
    viewModel.onMoveRequested();
});
```

Here, the View does not manually fetch battery level after each action. It simply reacts to changes.

This reduces boilerplate and eliminates explicit UI refresh logic.

The View is therefore reactive rather than imperative.

2.3.3 Behaviors and analysis

How MVVM Works (Operational Flow)

The interaction cycle in MVVM differs from MVC:

1. User interacts with View.
2. View forwards intent to ViewModel.
3. ViewModel invokes Model operations.
4. Model updates state.
5. ViewModel updates observable properties.
6. View automatically reacts to state change.

The key distinction lies in step 6: synchronization is automatic.

This reactive loop reduces the risk of UI inconsistency and simplifies state management in complex interfaces.

Architectural Implications

MVVM shifts architectural emphasis from event-driven control flow to state-driven rendering.

In MVC, the Controller orchestrates UI updates. In MVVM, UI updates emerge from observable state changes.

This has several consequences:

1. Synchronization logic becomes centralized.
2. Controllers are eliminated or significantly reduced.
3. The ViewModel must carefully avoid embedding domain rules.
4. State flows become easier to trace in unidirectional patterns.

However, MVVM introduces implicit behavior. Automatic updates can obscure the exact trigger chain, making debugging more complex compared to explicit MVC calls.

Thus, MVVM trades transparency for declarative convenience.

Analysis

Advantages

MVVM improves maintainability in UI-heavy applications because:

- It reduces repetitive update code.
- It minimizes the risk of stale UI state.
- It supports reactive rendering patterns.
- It integrates well with Android lifecycle management.

In mobile applications with configuration changes (screen rotation, activity recreation), ViewModels provide stability across lifecycle events, reducing state loss.

Disadvantages

However, MVVM introduces complexity:

1. Observable mechanisms add abstraction overhead.
2. Debugging reactive flows can be nontrivial.
3. Improper separation may cause ViewModels to absorb business logic.
4. Overuse of reactive streams may obscure control flow.

Additionally, in robotics or hardware-driven systems, explicit control sometimes provides clearer reasoning about timing and state transitions. Reactive updates may introduce subtle synchronization behavior if not carefully designed.

Therefore, MVVM improves UI state management but requires disciplined boundary enforcement to prevent architectural drift.

Critical Comparison: MVC vs MVVM

MVC emphasizes explicit coordination and structural separation. MVVM emphasizes reactive state synchronization.

MVC advantages:

- Transparent control flow
- Explicit update logic
- Easier reasoning about execution order

MVVM advantages:

- Reduced boilerplate
- Automatic UI synchronization
- Lifecycle-aware state handling

MVC risks:

- Fat Controllers

MVVM risks:

- Fat ViewModels
- Hidden reactive chains

2.4 Appendix D: ROS2 Overview

2.4.1 What is ROS2

ROS2 (Robot Operating System 2) is a middleware framework for building robot applications. It provides communication, tools, and libraries for composing complex robot systems from modular nodes.

2.4.2 Key concepts

Nodes

- Independent processes that perform specific tasks.
- Nodes communicate via topics, services, and actions.

Topics

- Publish/subscribe channels for streaming data.
- Designed for high-frequency sensor and state updates.

Services

- Synchronous request/response interactions.
- Used for operations like configuration or queries.

Actions

- Long-running goals with feedback and cancellation.
- Suitable for navigation and planning tasks.

Parameters

- Runtime configuration values stored on nodes.
- Can be read and set externally.

2.4.3 QoS (Quality of Service)

ROS2 exposes DDS QoS settings like reliability, durability, and history depth. Selecting the right QoS is critical in lossy networks or real-time systems.

2.4.4 Packages and workspaces

- A package is a unit of build and deployment.
- A workspace contains multiple packages and their build artifacts.

2.4.5 Lifecycle management

ROS2 supports managed nodes with lifecycle states (unconfigured, inactive, active, finalized). This helps bring up systems in a predictable order.

2.4.6 Why ROS2 for robotics

- Scalable communication infrastructure.
- Tooling for visualization and debugging.
- Large ecosystem of drivers and algorithms.

2.5 Appendix E: Distributed Systems

2.5.1 Overview

A distributed system is a collection of independent processes that cooperate via a network. Robotics stacks are inherently distributed, with sensors, controllers, and user interfaces running on separate nodes.

2.5.2 Key challenges

Latency and jitter

- Network delays vary over time.
- Control loops must tolerate nondeterministic timing.

Partial failures

- Components can fail independently.
- Systems should degrade gracefully when nodes disconnect.

Consistency

- Shared state is hard to keep synchronized.
- Stale data can lead to incorrect decisions.

2.5.3 Time and ordering

- Clocks drift between machines.
- Use timestamps and sequence numbers to detect ordering issues.

2.5.4 CAP tradeoff

The CAP theorem highlights tradeoffs between consistency, availability, and partition tolerance. Robotics systems often prioritize availability and timely responses over strict consistency.

2.5.5 Patterns

- Publish/subscribe for streaming state.
- Request/response for configuration.
- Heartbeats for liveness detection.

2.5.6 Applied to this project

Waypoint commands and odometry updates are exchanged over UDP, which trades reliability for low latency. The app must tolerate packet loss and occasional stale data while keeping the UI responsive.

2.6 Appendix F: Monolithic Architecture

2.6.1 Overview

A monolithic architecture bundles application components into a single deployable unit. This contrasts with microservices or distributed components.

2.6.2 Benefits

- Simple deployment and debugging.
- No network overhead between internal modules.
- Easier to maintain transaction boundaries.

2.6.3 Drawbacks

- Tightly coupled components reduce flexibility.
- Scaling requires duplicating the entire system.
- Large codebases can slow development without strong modularity.

2.6.4 Modular monolith

A modular monolith preserves a single deployment but enforces internal module boundaries. This approach often provides the best of both worlds when the system does not require independent scaling.

2.6.5 Applied to robotics tooling

A standalone UI app can be treated as a monolith with clear internal modules for networking, input handling, rendering, and data persistence. This keeps deployment simple while still enabling maintainability.

2.7 Appendix G: ROS2 Architecture

2.7.1 Layered structure

ROS2 is built as a stack of layers:

- Client libraries: rclcpp (C++), rclpy (Python)
- ROS client library: rcl
- Middleware interface: rmw
- DDS implementation: Fast DDS, Cyclone DDS, or Connext

2.7.2 Executors

Executors schedule callbacks for subscriptions, timers, and services. Common executors include single-threaded and multi-threaded variants. Callback groups control concurrency and prevent data races.

2.7.3 Composition

ROS2 supports composable nodes that run in a single process. This reduces overhead and improves performance when nodes are tightly coupled.

2.7.4 Lifecycle nodes

Lifecycle nodes provide deterministic startup and shutdown behavior. They move through states like unconfigured, inactive, active, and finalized.

2.7.5 DDS details

DDS handles discovery, serialization, and transport. QoS policies determine how data is delivered and what guarantees are provided.

2.7.6 Best practices

- Choose QoS profiles based on network conditions.
- Use namespaces to keep large systems organized.
- Keep topics small and publish at stable rates.

2.8 Appendix H: UDP

2.8.1 Overview

User Datagram Protocol (UDP) is a connectionless transport protocol that sends independent datagrams. It is fast and lightweight but provides no delivery or ordering guarantees.

2.8.2 Characteristics

- Connectionless and stateless.
- Packets can be dropped, duplicated, or reordered.
- Lower overhead than TCP.

2.8.3 Packet size and MTU

UDP payloads that exceed the path MTU may be fragmented, which increases loss risk. Design packet formats to be compact and avoid fragmentation where possible.

2.8.4 Checksums

UDP includes a checksum for error detection but no retransmission. Applications must decide whether to drop or tolerate corrupted packets.

2.8.5 Best practices for control systems

- Include sequence numbers or plan IDs to identify stale data.
- Use fixed-size binary formats for consistent parsing.
- Keep send rates bounded to avoid congestion.
- For critical commands, consider sending multiple times or adding acknowledgments.

2.8.6 Applied to target setting

The waypoint protocol favors low latency and simplicity. The app can tolerate occasional packet loss while maintaining responsiveness.

2.9 Appendix I: Quaternions

2.9.1 Overview

Quaternions represent 3D rotations without gimbal lock. A quaternion is written as $q = (w, x, y, z)$ where w is the scalar component and (x, y, z) is the vector component.

2.9.2 Unit quaternions

A rotation quaternion is unit length:

$$\|q\| = \sqrt{w^2 + x^2 + y^2 + z^2} = 1$$

2.9.3 Quaternion multiplication

Combining rotations uses quaternion multiplication, which is not commutative. If q_1 and q_2 are rotations, the composed rotation is $q = q_2 otimes q_1$.

2.9.4 Axis-angle conversion

For rotation angle $theta$ around unit axis $hat{u}$:

$$q = (\cos(\theta/2),\ \hat{u}_x \sin(\theta/2),\ \hat{u}_y \sin(\theta/2),\ \hat{u}_z \sin(\theta/2))$$

2.9.5 Yaw extraction (2D robots)

For planar motion, yaw can be extracted from a quaternion:

$$\text{yaw} = \arctan 2(2(wz + xy),\ 1 - 2(y^2 + z^2))$$

2.9.6 Numerical stability

- Normalize quaternions after integration.
- Avoid repeated small-angle conversions without renormalization.

2.9.7 Applied to odometry

Yaw is computed from quaternion orientation and then used in frame transforms for velocities and positions.

2.10 Appendix J: Kinematics

2.10.1 Overview

Kinematics describes the relationship between motion variables without considering forces. For robots, this links wheel velocities to body motion.

2.10.2 Forward kinematics

Given wheel speeds, compute body velocities. For a holonomic robot, the relationship can be expressed as:

$$\mathbf{v} = \mathbf{J} \cdot \omega$$

where $mathbf{J}$ is the kinematic matrix and $mathbf{omega}$ are wheel rates.

2.10.3 Inverse kinematics

Given desired body velocities, compute wheel speeds:

$$\omega = \mathbf{J}^{-1} \cdot \mathbf{v}$$

If $mathbf{J}$ is not square, a pseudoinverse is used.

2.10.4 Coordinate frames

- Local frame: attached to the robot body.
- Global frame: fixed to the environment or map.

Transform between frames using rotation matrices derived from yaw.

2.10.5 Discretization

Velocities are integrated over time to compute position:

$$x_{k+1} = x_k + v_x \Delta t$$

and similarly for y and $theta$.

2.10.6 Applied to this system

The kinematic model converts desired planar velocities into wheel velocities and back into odometry updates.

2.11 Appendix K: State Machine

2.11.1 Overview

A state machine models a system with a finite number of modes. Each state has well-defined behavior and transitions that occur on events or conditions.

2.11.2 Key terms

- State: a named mode of operation.
- Transition: a change from one state to another.
- Guard: a condition that must be true for a transition.
- Action: logic executed on entry, exit, or during a state.

2.11.3 Design guidelines

- Keep states orthogonal: one reason to change.
- Prefer explicit transitions over implicit side effects.
- Log transitions for debugging.

2.11.4 Hierarchical state machines

Nested states allow shared behavior at higher levels. This reduces duplication when multiple states share common logic.

2.11.5 Applied to navigation

The navigation pipeline can be modeled as:

- IDLE: waiting for a waypoint.
- NAVIGATING: moving toward a target.
- EDIT: applying a waypoint update.
- PAUSED: holding position.
- RETURNING: going back to a visited waypoint.

This structure simplifies reasoning about behavior and failure handling.

2.12 Appendix L: PID Control

2.12.1 Overview

PID control combines proportional, integral, and derivative terms to reduce error between a desired value and the measured value.

2.12.2 PID equation

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

2.12.3 Discrete form

$$u_k = K_p e_k + K_i \sum_{i=0}^{k} e_i \Delta t + K_d \frac{e_k - e_{k-1}}{\Delta t}$$

2.12.4 Tuning

- K_p sets responsiveness but can overshoot.
- K_i removes steady-state error but can wind up.
- K_d dampens oscillation but amplifies noise.

2.12.5 Anti-windup

Clamp the integral term when actuators saturate:

$$I_k = \text{clamp}(I_k, I_{min}, I_{max})$$

2.12.6 Filtering

Derivative terms often use low-pass filtering to reduce noise.

2.12.7 Applied to robot motion

The controller uses PD components on position and yaw with filtered derivative terms for smoother navigation.

2.13 Appendix M: Localization

2.13.1 Overview

Localization estimates a robot's position and orientation in a global frame. It combines motion models with sensor measurements.

2.13.2 Sources of localization

- Odometry: integrates wheel or motor motion.
- IMU: measures angular velocity and acceleration.
- Visual markers or lidar: provide absolute corrections.

2.13.3 Probabilistic view

State estimation often uses a probabilistic model:

$$x_k = f(x_{k-1}, u_k) + w_k z_k = h(x_k) + v_k$$

where w_k and v_k represent process and measurement noise.

2.13.4 Filters

- Extended Kalman Filter (EKF): linearizes nonlinear models.
- Particle Filter: represents state as particles.

2.13.5 Frames

Common frames in robotics include:

- base_link: robot body.
- odom: local frame for short-term accuracy.
- map: global reference frame.

2.13.6 Applied to this system

Odometry provides short-term pose estimates. For long-term accuracy, periodic corrections can be applied using external references.

2.14 Appendix N: Odometry

2.14.1 Overview

Odometry estimates robot motion by integrating velocity over time. It is fast and local but accumulates drift due to wheel slip and noise.

2.14.2 Kinematic integration

Given linear velocities in the global frame:

$$x_{k+1} = x_k + v_x \Delta t y_{k+1} = y_k + v_y \Delta t$$

Yaw integration uses angular velocity:

$$\theta_{k+1} = \theta_k + \omega_z \Delta t$$

2.14.3 Error sources

- Wheel slip or uneven terrain.
- Encoder quantization.
- Incorrect wheel radius or geometry parameters.

2.14.4 Mitigation

- Periodic calibration of wheel parameters.
- Fusing odometry with IMU or external sensors.
- Avoiding long open-loop runs without correction.

2.14.5 Applied to this project

Odometry is derived from motor velocities and transformed into the global frame for display and control.

2.15 Appendix O: Waypoint Navigation

2.15.1 Overview

Waypoint navigation drives a robot through a sequence of target positions. Key components include path planning, control, and arrival detection.

2.15.2 Path planning

- For sparse waypoints, straight-line segments are common.
- For smoother trajectories, use splines or polynomial paths.

2.15.3 Arrival detection

A waypoint is considered reached when distance and timing constraints are met:

$$\text{dist} = \sqrt{(x_t - x)^2 + (y_t - y)^2}$$

If dist is below a tolerance for a specified duration, the waypoint is complete.

2.15.4 Handling edits

When waypoints are edited, smoothing can reduce abrupt changes in command velocity. A simple low-pass update is:

$$x_{new} = (1 - \alpha)x_{old} + \alpha x_{update}$$

2.15.5 Return behavior

Visited waypoints can be stored in a stack to support a return-to-base function.

2.15.6 Applied to this system

The app sends waypoint lists and updates to the robot. The controller processes the queue, updates the active target, and transitions between states.

2.16 Appendix P: Complementary Filtering

2.16.1 Overview

A complementary filter combines two signals with different frequency qualities. One signal provides good low-frequency behavior, the other provides high-frequency response. The sum provides a balanced estimate.

2.16.2 Basic equation

$$y = \alpha y_{fast} + (1 - \alpha)y_{slow}$$

where $0 < alpha < 1$.

2.16.3 Why it works

- The low-pass component reduces noise and drift.
- The high-pass component preserves responsiveness.

2.16.4 Discrete implementation

```
filtered = alpha * new_value + (1 - alpha) * previous_filtered
```

2.16.5 Applied to derivative terms

Derivative signals are noisy. A complementary filter can smooth the derivative term in a PD or PID controller, improving stability without a large delay.

2.16.6 Applied to waypoint updates

When target updates occur mid-motion, filtering avoids sudden jumps in desired position and keeps the control output smooth.

2.17 Appendix Q: CAN Bus

2.17.1 Overview

Controller Area Network (CAN) is a robust bus protocol used in automotive and robotic systems for reliable device communication.

2.17.2 Frame structure

- Identifier: arbitration and message priority.
- Data length: number of bytes (0 to 8 for classic CAN).
- Data: payload bytes.
- CRC: error detection.

2.17.3 Arbitration

CAN uses non-destructive bitwise arbitration. The lowest identifier wins bus access without corrupting messages.

2.17.4 Bitrate and timing

All nodes must share the same bitrate and timing parameters. Mismatched timing causes errors and bus-off states.

2.17.5 Error handling

CAN includes error counters and automatic retransmission. Persistent errors can place a node into a bus-off state until recovery.

2.17.6 Classic CAN vs CAN FD

- Classic CAN: 8-byte payload limit.
- CAN FD: larger payload and higher data phase bitrate.

2.17.7 Applied to robotics

CAN is often used for motor controllers and sensor modules that require robust communication in noisy electrical environments.

CHAPTER

THREE

INDICES AND TABLES

- genindex
- modindex
- search

BIBLIOGRAPHY

[[Ref]1] Build a responsive ui with constraintlayout. Accessed: 2026-02-13. URL: https://developer.android.com/develop/ui/views/layout/constraint-layout.

[[Ref]2] Coordinates and transformations. Accessed: 2026-02-15. URL: https://motion.cs.illinois.edu/RoboticSystems/CoordinateTransformations.html.

[[Ref]3] Mvc framework – architecture. Accessed: 2026-02-13. URL: https://www.tutorialspoint.com/mvc_framework/mvc_framework_architecture.htm.

[[Ref]4] Model-view-viewmodel (mvvm). Accessed: 2026-02-13. URL: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm.

[[Ref]5] Ros 2: what is dds? Accessed: 2026-02-15. URL: https://community.rti.com/page/ros-2-what-dds.

[[Ref]6] Unidirectional link vs bidirectional link vs loopback link. May 2021. Accessed: 2026-02-13. URL: https://ccna-classes.com/ccna-study-resources/unidirectional-link-vs-bidirectional-link-vs-loopback-link/.

[[Ref]7] Interfaces. 2026. ROS 2 Documentation: Humble. URL: http://docs.ros.org/en/humble/Concepts/Basic/About-Interfaces.html.

[[Ref]8] Nodes. 2026. ROS 2 Documentation: Humble. URL: https://docs.ros.org/en/humble/Concepts/Basic/About-Nodes.html.

[[Ref]9] Topics. 2026. ROS 2 Documentation: Humble. URL: https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html.

[[Ref]10] A. Boghossian, J. Brown, and S. Zak. P, i, d, pi, pd, and pid control. Accessed: 2026-02-16. URL: https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Chemical_Process_Dynamics_and_Controls_(Woolf)/09:_Proportional-Integral-Derivative_(PID)_Control/9.02:_P_I_D_PI_PD_and_PID_control.

[[Ref]11] P. L. Dordal. Udp transport. 2012. Accessed: 2026-02-14. URL: https://intronetworks.cs.luc.edu/current/html/udp.html.

[[Ref]12] D. C. Karnopp, D. L. Margolis, and R. C. Rosenberg. *System Dynamics: Modeling, Simulation, and Control of Mechatronic Systems*. John Wiley & Sons, 2012.

[[Ref]13] Huseyin Kutluca. Robot operating system 2 (ros 2) architecture. December 2020. Accessed: 2026-02-15. URL: https://medium.com/software-architecture-foundations/robot-operating-system-2-ros-2-architecture-731ef1867776.

[[Ref]14] B. P. Lathi. *Principles of Linear Systems and Signals*. Oxford University Press, 2 edition, 2009.

[[Ref]15] E. A. Lee and P. P. Varaiya. *Structure and Interpretation of Signals and Systems*. Pearson Education, 2003.

[[Ref]16] S. Lipschutz and M. L. Lipson. *Schaum's Outline of Discrete Mathematics*. McGraw-Hill, 3 edition, 2007.

[[Ref]17] A. Nandwanya. Constraintlayout, demystified – how it really works and when you should reach for it. September 2025. Accessed: 2026-02-12. URL: https://androidengineers.substack.com/p/constraintlayout-demystified-how.

[[Ref]18] S. Narayana. Cs 352 user datagram protocol. Accessed: 2026-02-14. URL: https://people.cs.rutgers.edu/~sn624/352-S21/lectures/transport-udp-error-detection.pdf.

[[Ref]19] H. P. Oliveira and others. Dynamic models for omni-directional robots with 3 and 4 wheels. In *ICINCO*. 2008.

[[Ref]20] A. Phunopas and S. Inoue. Motion improvement of four-wheeled omnidirectional mobile robots for indoor terrain. *Journal of Robotics, Networking and Artificial Life*, 4(4):275–282, 2018.

[[Ref]21] A. Silberschatz, P. B. Galvin, and G. Gagne. *Operating System Concepts*. Wiley, 10 edition, 2018.

[[Ref]22] S. Simrock. Control theory. Accessed: 2026-02-16. URL: https://cds.cern.ch/record/1100534/files/p73.pdf.

[[Ref]23] S. Singh. Mvc framework: a modern web application development approach and working. *International Research Journal of Engineering and Technology (IRJET)*, 7(1):51–55, January 2020. URL: https://www.irjet.net/archives/V7/i1/IRJET-V7I111.pdf.

[[Ref]24] J. Verschelde. Client/server interactions. 2017. Accessed: 2026-02-14. URL: https://homepages.math.uic.edu/~jan/mcs275/mcs275notes/lec27.html.

[[Ref]25] Cisco Networking Academy. *CCNAv7: Introduction to Networks Companion Guide*. Cisco Press, Indianapolis, IN, USA, 2020.

[[Ref]26] K. J. Åström. Control system design. 2002. Accessed: 2026-02-16. URL: https://www.cds.caltech.edu/~murray/courses/cds101/fa02/caltech/astrom-ch6.pdf.